CSE557 · BIG DATA ANALYTICS · 5TH SEM

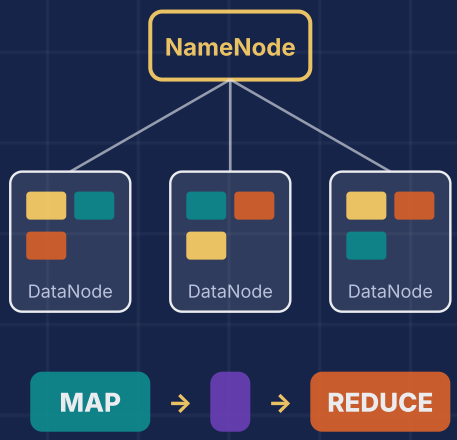
Unit 2
Study Guide

Data Analytics with Hadoop — every slide from Lectures 1–3, explained simply, with diagrams, tips and solved questions.

1 **Basics of Hadoop**
Data! · Storage & analysis · Data format · MapReduce · Scaling out · Data flow

2 **The Hadoop Distributed File System**
Design · Blocks · NameNode/DataNode · Federation · HA · File read & write

3 **Developing a MapReduce Application**
Workflows · MRUnit · Job run · MR1 · YARN · Failures · Scheduling · Shuffle & sort



Textbook: Tom White, *Hadoop: The Definitive Guide* (O'Reilly)

CIE-1 Edition

How to use this guide

Unit 2 is about **how Hadoop actually works**: how it **stores** huge files across many machines (HDFS) and how it **processes** them in parallel (MapReduce, YARN). Read in order — Chapter 1 gives the big picture, Chapters 2 and 3 zoom into storage and processing.

What the coloured boxes mean

🔑 **KEY POINT**
Must-know fact. If you remember nothing else, remember these.

🎯 **EXAM TIP**
How this topic is usually asked and how to score full marks.

💡 **TIP**
A shortcut to understand or study the idea faster.

🧠 **MEMORY TRICK**
Mnemonics to recall lists and step orders quickly.

⚠️ **COMMON MISTAKE**
Things students mix up — avoid losing marks here.

📌 **EXAMPLE**
A real-life example that makes the concept click.

Also: yellow highlighted text = exam keywords. Numbered badges like 3 match the step numbers in the textbook diagrams.

Suggested study plan (about 7–8 hours total)

Session 1
Chapter 1 — Hadoop basics, MapReduce example
≈ 2 hrs

Session 2
Chapter 2 — HDFS, file read & write
≈ 2 hrs

Session 3
Ch 3 Part A — Workflows, MRUnit, MR1, YARN
≈ 2 hrs

Session 4
Ch 3 Part B — Failures, Scheduling, Shuffle + Cheat sheet
≈ 1.5 hrs

💡 **HOW TO STUDY THE "STEP" TOPICS**

Unit 2 has many **step-by-step processes** (file read: 6 steps, file write: 7 steps, MR1 job run: 10 steps, YARN job run: 11 steps). For each one:

1. First learn the **actors** (who is involved — client, NameNode, DataNode...).
2. Then trace the **diagram with your finger**, saying each numbered arrow out loud.
3. Then close the book and **redraw the diagram** from memory. If you can draw it, you can write the answer.

🎯 **ABOUT CIE-1**

CIE-1 (30 marks) covers CO1–CO3: **Unit 1, Unit 2 and the first part of Unit 3**. Unit 2 is the **most diagram-heavy** unit — "Anatomy of a file read/write", "Classic MapReduce vs YARN", "Shuffle and sort" and "HDFS architecture" are classic 10-mark questions. Always write: **definition → diagram → numbered steps → example**.

Contents

Chapter 1 · Basics of Hadoop 4

1.1 Data! 4

1.2 Data storage and analysis 5

1.3 What is Hadoop? (history, components, ecosystem) 7

1.4 Data format — the NCDC weather dataset 9

1.5 Analyzing the data with Hadoop — Map & Reduce 10

1.6 Java MapReduce — Mapper, Reducer, Driver 11

1.7 Scaling out & data flow 14

Chapter 2 · The Hadoop Distributed File System 17

2.1 Distributed file systems 17

2.2 The design of HDFS 18

2.3 HDFS concepts — Blocks 19

2.4 NameNodes and DataNodes 20

2.5 HDFS Federation 21

2.6 HDFS High Availability 22

2.7 Data flow — Anatomy of a file read 23

2.8 Data flow — Anatomy of a file write 25

Chapter 3 · Developing a MapReduce Application 28

3.1 MapReduce workflows & JobControl 28

3.2 Writing a unit test with MRUnit 29

3.3 Anatomy of a MapReduce job run 31

3.4 Classic MapReduce (MapReduce 1) 32

3.5 YARN (MapReduce 2) 34

3.6 Failures 37

3.7 Job scheduling 39

3.8 Shuffle and sort 41

Final Revision 45

Cheat sheet (all 3 chapters) 45

All memory tricks in one place 46

Important exam questions 48

CHAPTER 1

Basics of Hadoop

What you'll learn

1. How much data the world creates
2. Why bigger disks didn't solve it
3. What Hadoop is & its history
4. HDFS + MapReduce + YARN in brief
5. The NCDC weather data format
6. Map → Shuffle → Reduce, step by step
7. Mapper, Reducer & Driver in Java
8. Splits, data locality & data flow

⏱ ≈ 2 hours | Slides: Lecture 1 (pp. 1–66) | Tom White Ch. 1–2

1.1 Data!

🗣 IN SIMPLE WORDS

We live in the **data age**. Every click, photo, message and sensor reading creates data. There is so much of it that **one computer can no longer store or analyse it** — that's the problem Hadoop solves.

How big is Big Data?

- In **2006** the "digital universe" was about 0.18 zettabytes.
- By **2011** it was forecast to be 1.8 zettabytes — a **10× growth**.
- **1 zettabyte = 10^{21} bytes** = 1,000 exabytes = 1 million petabytes = 1 billion terabytes.
- To imagine it: like **every person on Earth carrying a disk drive**.

From terabytes to zettabytes (each ×1,000)



1 ZB = 10^{21} bytes = 1 billion TB

Real-world examples (from your slides / textbook)

Organisation	How much data
New York Stock Exchange	≈ **1 terabyte** of new trade data **per day**
Facebook	≈ **10 billion photos** → about **1 petabyte** of storage
Ancestry.com (genealogy site)	≈ **2.5 petabytes** of data
Internet Archive	≈ **2 petabytes**, growing by **20 terabytes per month**
Large Hadron Collider (Geneva)	≈ **15 petabytes** of data **per year**

🤖 Machines may generate even more data

Many small streams become Big Data: machine logs · RFID · sensors · GPS · retail transactions · email + more. Machine data can be larger than people's data.

📷 MyLifeBits — archiving a whole life

A Microsoft research project: store a photo **every minute**, all **phone calls**, **emails** and **documents** of one person → about 1 GB of data per month. Add continuous audio/video and it becomes much larger.

😀 The good news

Big Data is here. More data → more opportunities to discover patterns and value. ("More data usually beats better algorithms.")

😟 The bad news

How do we store and analyse it? One machine eventually becomes **too small or too slow**.

🔑 BRIDGE TO HADOOP

The answer is to distribute storage + distribute processing across many machines. That is exactly what Hadoop does.

1.2 Data storage and analysis

🗣 IN SIMPLE WORDS

Hard disks got **much bigger** over the years, but they did **not get much faster** at reading data. So reading a full modern disk takes hours. The challenge is not only *storing* more — it is **reading and processing** more.

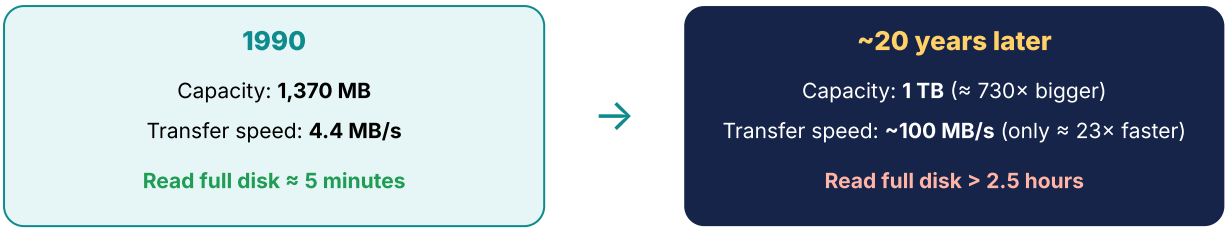


Figure 1.1 — Disk capacity grew much faster than disk speed: bigger capacity ≠ proportionally faster access

The obvious idea: read from many disks at once

1 TB total data → **÷ 100** drives → **10 GB** per drive

If 100 drives each hold 1/100th of the data and work **in parallel**, we can read the whole 1 TB in under 2 minutes instead of 2.5 hours.

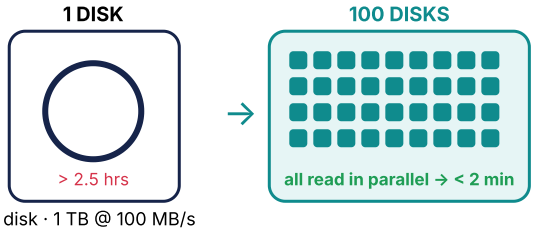


Figure 1.2 — Parallel reading changes the waiting time

PARALLELISM

Divide a large job into smaller pieces and do them **simultaneously**.

💡 **"BUT AREN'T 100 DISKS WASTEFUL?"**

No — the 100 disks can hold **many datasets** (Dataset 1, 2, 3, 4, 5...). Many users **share the cluster** → shorter analysis times and better hardware utilisation.

Parallel disks create two new problems

Problem 1 · Hardware can fail

More machines → more chances that one disk, node or component fails. **Need: reliability.**

✅ **Solution — Replication:** keep **redundant copies** of the data (Copy 1, Copy 2, Copy 3) so another copy can be used if hardware fails. HDFS uses replication.

Problem 2 · Data must be combined

Results from many disks must be **mixed and combined** correctly. **Need: distributed analysis.**

✅ **Solution — MapReduce:** a simpler programming model. Data from all disks → **MAP** → **shuffle** (bring keys + values together) → **REDUCE** → final result. Map + Reduce hide much of the distributed coordination complexity.

1.3 What is Hadoop?

DEFINITION

Hadoop is an open-source framework that allows us to store and process large data sets in a parallel and distributed manner. In a nutshell: **Hadoop = reliable shared storage + reliable distributed analysis.**

- Hadoop is basically written in **Java**.
- Created by Doug Cutting and Mike Cafarella.
- It contains **three core components**:
  1. **HDFS** — the primary data storage system used by Hadoop applications → **storage layer**.
  2. **MapReduce** — the processing unit of Hadoop → **processing layer**.
  3. **YARN** — **resource management layer**.
- Plus **Hadoop Common** — shared utilities/libraries used by the other modules.

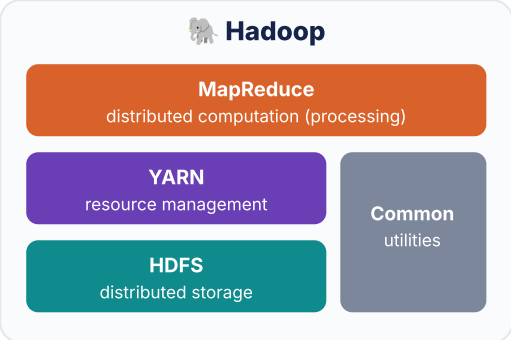


Figure 1.3 — Core components of Hadoop

MEMORY TRICK

"Hadoop has a HMY body" → **H**DFS = the **storage** (stomach), **M**apReduce = the **processing** (brain), **Y**ARN = the **manager** who decides who gets resources.

A brief history of Hadoop

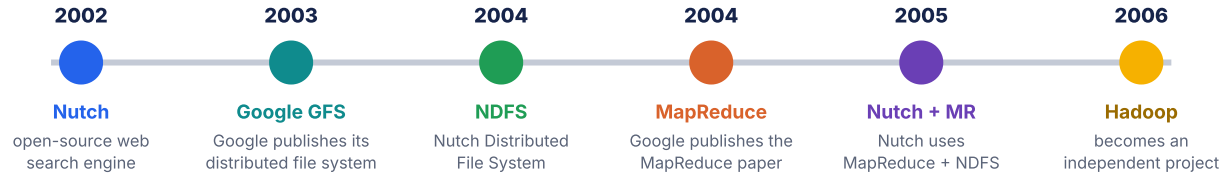


Figure 1.4 — The journey to Hadoop: Nutch search problem → distributed storage → distributed processing → Hadoop

- By **2005** Nutch was using MapReduce and NDFS. In **February 2006** they became a **separate project called Hadoop**.
- **Why "Hadoop"?** The name is not an acronym. Doug Cutting named it after his **child's stuffed yellow toy elephant** — short, distinctive and easy to remember.
- **2006 onward — Yahoo! helps Hadoop grow:** Yahoo!'s Grid team adopted Hadoop and provided people, clusters and real-world scale: May 2006 → 300 nodes · Oct 2006 → 600 · Jan 2007 → 900 · Apr 2007 → 1,000 · Mar 2009 → **24,000 nodes**.

When	Milestone	Detail
Jan 2008	Apache top-level project	Hadoop becomes a major Apache project
Apr 2008	1 TB sort	Sorted 1 terabyte in **209 seconds on 900 nodes** (world record)
Mar 2009	24,000 nodes	17 Yahoo! clusters in total
Apr 2009	500 GB / 59 sec	Minute-sort result on 1,400 nodes

The story changed from "Can this work?" to "How large can this scale?"

Why should we use Hadoop?

🏆 Popular

Captured ~**90% of the Big Data market**; code contributed by IBM, Cloudera, Yahoo!.

📈 Scalable

Add more cheap machines to grow storage & processing.

🛡 Fault tolerant

Replication keeps data safe when machines fail.

🧩 Flexible with data

Handles structured, semi-structured & unstructured data.

The Hadoop ecosystem

Around the core (HDFS, YARN, MapReduce) there are many related projects. Your slide shows these:

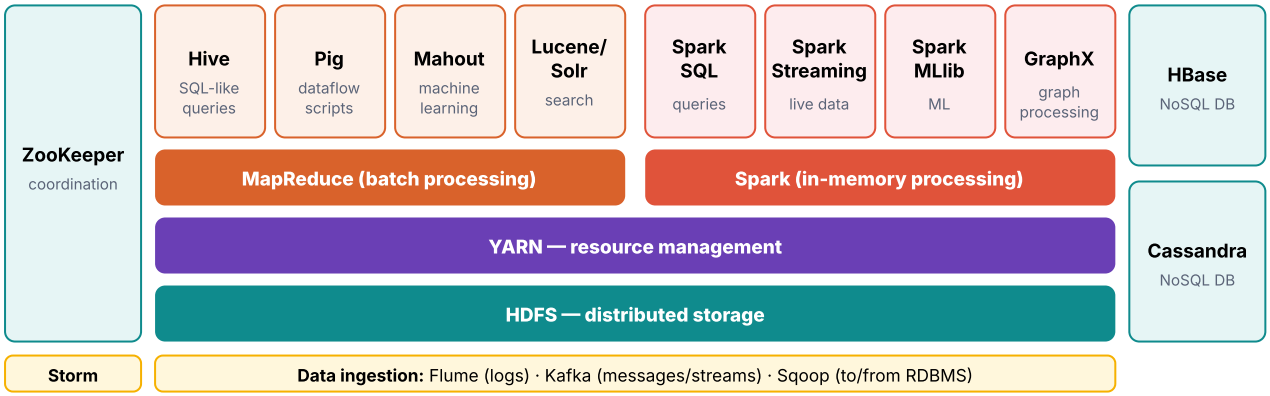


Figure 1.5 — The Hadoop ecosystem (layers are simplified)

HDFS and MapReduce — first look

🗄 HDFS — Hadoop Distributed File System

- Has a master–slave architecture.
- Data files are **divided into multiple blocks**.
- Blocks are stored in a distributed manner on the cluster of **slave nodes**; the **master** stores all the **metadata**.
- HDFS has two daemons: **NameNode** (master) and **DataNode** (slave).

Full details in Chapter 2.

⚙️ MapReduce

- The data processing layer of Hadoop.
- A programming model for processing large datasets by doing work **in parallel**.
- Processes data in two phases: **Map phase** and **Reduce phase**.
- **MAP**: read pieces of data, produce key–value pairs → **SHUFFLE**: bring values with the same key together → **REDUCE**: combine each group into the final result.

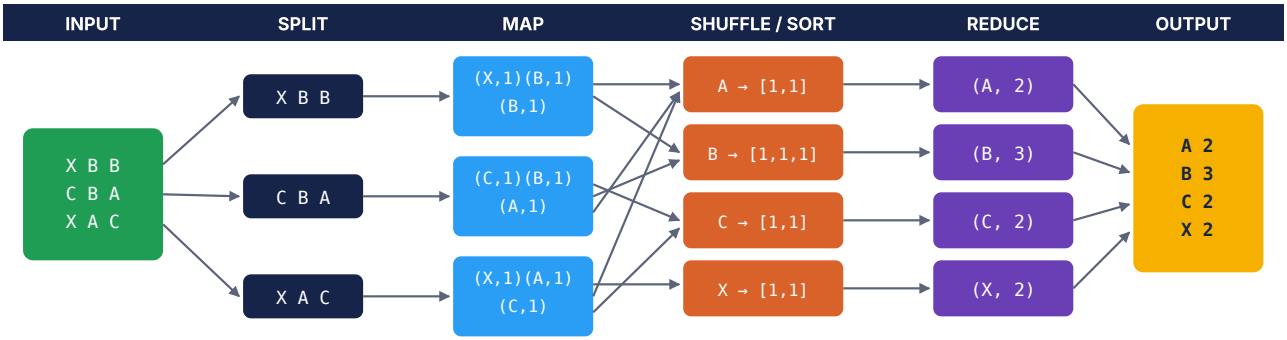


Figure 1.6 — MapReduce counting letters: input → split → map (emit (letter,1)) → shuffle/sort (group by key) → reduce (add up) → output

1.4 Data format — the NCDC weather dataset

🗣 IN SIMPLE WORDS

To learn MapReduce, the textbook uses **weather data**. Thousands of weather stations record readings **hour after hour** for many years → a huge dataset. The question we'll answer: **"What is the highest temperature recorded in each year?"**

- Data comes from the **National Climatic Data Center (NCDC)**.
- The data is semi-structured and record-oriented — each line is **one weather observation**. That makes it a good fit for MapReduce.
- Each record is stored in a **line-oriented ASCII format** with **fixed-width fields** (each field is always at the same character position).

One record — fields we care about

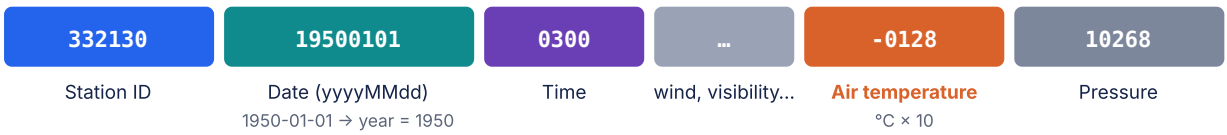


Figure 1.7 — A weather record is a long line of fixed-width fields

📌 WHY DOES THE RECORD SAY -0128?

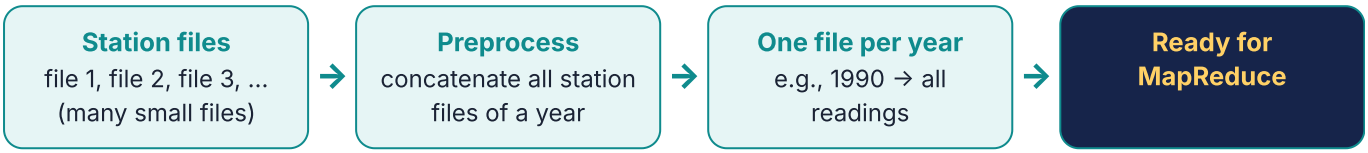
Temperature is stored as **degrees Celsius × 10**.

`-0128 ÷ 10 = -12.8 °C`

The program extracts and interprets this fixed-width temperature field. Missing readings are stored as **9999**.

📌 HOW THE FILES ARE ORGANISED

Originally: one directory per **year** (e.g., `raw/1990/`) containing a **gzipped file for each weather station** (`010010-99999-1990.gz`, ...) — **tens of thousands of small files**.



🔑 KEY POINT

Hadoop works better with a small number of large files than with many small files. Preprocessing changes only the **file organisation** — not the meaning of the weather records.

1.5 Analyzing the data with Hadoop — Map & Reduce

MAP AND REDUCE

- MapReduce works by breaking the processing into two phases: the map phase and the reduce phase.
- Each phase has key–value pairs as input and output, the types of which may be chosen by the programmer.
- The programmer also specifies **two functions**: the **map function** and the **reduce function**.

📌 THE TASK

Find the **maximum temperature for each year**. Example input: `1990 → 25°C, 31°C, 28°C` and `1991 → 27°C, 34°C, 30°C`. Wanted output: `1990 → 31°C` | `1991 → 34°C`.

Step 1 · MAP — extract only what we need

The mapper reads each long weather record (station + date + time + wind + visibility + temperature + pressure + ...) and emits just **(year, temperature)**. It also **filters out bad or missing records**.

`(1990, 25) (1990, 31) (1990, 28) (1991, 27) …` → **Key = Year | Value = Temperature**

Input to map: key = **line offset** in the file (ignored), value = **the line of text** (the record).

Step 2 · SHUFFLE & SORT — group the same years (done automatically by Hadoop)

Before reduce, the framework **sorts and groups** values having the same key:
`1990 → [25, 31, 28]` `1991 → [27, 34, 30]` (textbook: `1949 → [111, 78]`, `1950 → [0, 22, −11]`)

Step 3 · REDUCE — find the maximum in each group

The reduce function receives **(key, list of values)** and iterates through the list picking the maximum:
`MAX(111, 78) = 111` `MAX(0, 22, −11) = 22` → final output `(1949, 111)`, `(1950, 22)`.

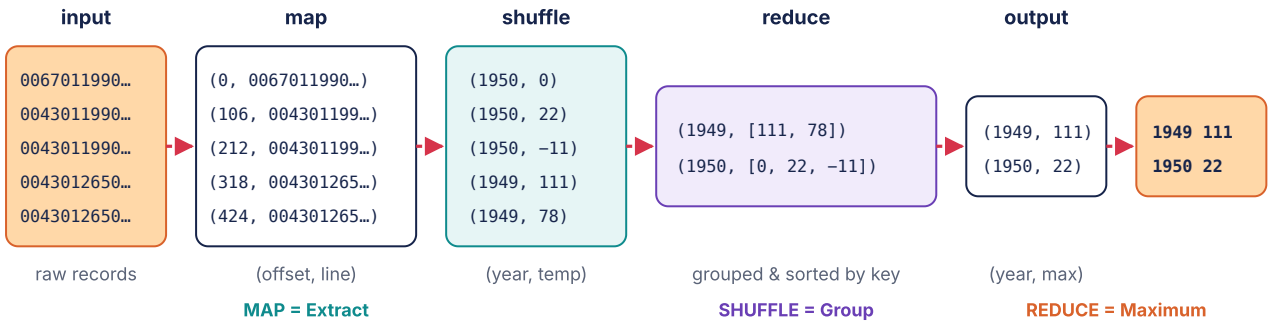


Figure 1.8 — MapReduce logical data flow for the maximum-temperature example (Textbook Fig. 2-1)

🔑 WHO DOES WHAT?

The **programmer** writes only two things: the **map function** ("for every weather record, extract the year and temperature") and the **reduce function** ("for every year, find the maximum temperature"). Hadoop takes care of distributing the work, grouping the intermediate key–value pairs, and passing them to reducers.

🧠 MEMORY TRICK

MAP = Extract · SHUFFLE = Group · REDUCE = Summarise — "**E G S**: **E**very **G**roup **S**ummarised".

1.6 Java MapReduce — Mapper, Reducer, Driver

To run a MapReduce program, we need **three things**:

1 · Map function (Mapper class)

Reads each input record (one line of weather data), extracts year & temperature, **filters out bad/missing records**, outputs intermediate key–value pairs `<Year, Temperature>`.

2 · Reduce function (Reducer class)

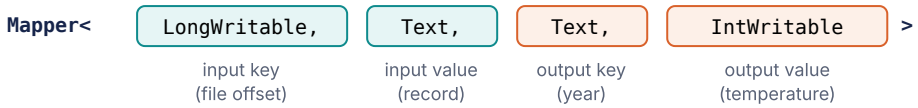
Receives a key and a **list of all values** (a year and all its temperatures), finds the maximum, outputs `<Year, Max Temperature>`.

3 · Code to run the job (Driver / Main class)

Configures the job (name, input & output paths), specifies the Mapper and Reducer classes, sets the key and value types, **submits the job** to Hadoop and waits for it to finish.

In short: a MapReduce program has a Mapper, a Reducer, and a Driver to run the job.

The Mapper




```
public class MaxTemperatureMapper
    extends Mapper<LongWritable, Text, Text, IntWritable> {
  private static final int MISSING = 9999;
  @Override
  public void map(LongWritable key, Text value, Context context)
      throws IOException, InterruptedException {
    String line = value.toString();
    String year = line.substring(15, 19);           // year is at characters 15–19
    int airTemperature;
    if (line.charAt(87) == '+') {                   // parseInt doesn't like leading plus signs
      airTemperature = Integer.parseInt(line.substring(88, 92));
    } else {
      airTemperature = Integer.parseInt(line.substring(87, 92));
    }
    String quality = line.substring(92, 93);
    if (airTemperature != MISSING && quality.matches("[01459]")) {  // skip bad/missing readings
      context.write(new Text(year), new IntWritable(airTemperature));
    }
  }
}
```

The Reducer

`Reducer<Text, IntWritable, Text, IntWritable>` = <Year, Temperatures, Year, Maximum Temperature>. The reducer's **input types must match the mapper's output types**.

```
public class MaxTemperatureReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {
  @Override
  public void reduce(Text key, Iterable<IntWritable> values, Context context)
      throws IOException, InterruptedException {
    int maxValue = Integer.MIN_VALUE;
    for (IntWritable value : values) {
      maxValue = Math.max(maxValue, value.get());      // keep the largest
    }
    context.write(key, new IntWritable(maxValue));
  }
}
```

The Driver (application that runs the job)

```
public class MaxTemperature {
  public static void main(String[] args) throws Exception {
    if (args.length != 2) {
      System.err.println("Usage: MaxTemperature <input path> <output path>");
      System.exit(-1);
    }
    Job job = new Job();
    job.setJarByClass(MaxTemperature.class);          // Hadoop finds the JAR to ship to the cluster
    job.setJobName("Max temperature");
    FileInputFormat.addInputPath(job, new Path(args[0]));   // input path
    FileOutputFormat.setOutputPath(job, new Path(args[1])); // output dir (must NOT exist)
    job.setMapperClass(MaxTemperatureMapper.class);
    job.setReducerClass(MaxTemperatureReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    System.exit(job.waitForCompletion(true) ? 0 : 1);   // submit and wait
  }
}
```

💡 **HADOOP DATA TYPES**

Hadoop uses its own types optimised for network serialisation: `LongWritable` ≈ Java `long`, `Text` ≈ Java `String`, `IntWritable` ≈ Java `int`.

A test run

After writing the Mapper, Reducer and Driver, don't immediately run them on years of data. First run on a **very small sample containing only 5 weather records**:

```
% export HADOOP_CLASSPATH=hadoop-examples.jar
% hadoop MaxTemperature input/ncdc/sample.txt output
```

Hadoop prints a log: *Running job: job_local_0001 … map 100% reduce 100% … Job completed successfully*, followed by **counters**, e.g.:

- Map input records = **5**, Map output records = **5**
- Reduce input groups = **2** (two years), Reduce input records = 5
- Reduce output records = **2**

```
% cat output/part-r-00000
1949    111
1950    22
```

The output file is named `part-r-00000` (r = reducer output, 00000 = reducer number). 111 = 11.1°C and 22 = 2.2°C.

📌 **LAB STEPS FROM YOUR SLIDES (RUNNING IT YOURSELF)**

1. Check the bash file created in the Hadoop installed folder.
2. Run it: `source bash.sh`
3. Check Hadoop: `hadoop`
4. Create a folder `weather` inside the Hadoop folder.
5. Make `driver.java`, `mapper.java`, `reducer.java`.
6. Compile: `javac -d . *.java`
7. Set main class: `echo Main-Class: weather.driver > Manifest.txt`
8. Create JAR: `jar cfm weather.jar Manifest.txt weather/*.class`
9. Create input file `input.txt` (download weather input).
10. Run: `hadoop jar weather.jar input.txt output`
11. See output: `cat output/*`

1.7 Scaling out & data flow

IN SIMPLE WORDS

On real (big) data, Hadoop cuts the input into **pieces called splits**, runs **one map task per split** on many machines at the same time — preferably on the machine that **already holds that piece of data** — and then sends the map results to the reducers.

MAPREDUCE JOB

A unit of work that the client wants to be performed. It consists of the **input data**, the **MapReduce program**, and **configuration information**.

- Hadoop runs the job by dividing it into **tasks** of two types: map tasks and reduce tasks.
- Tasks are **scheduled using YARN** and run on nodes in the cluster. If a task fails, it is **automatically rescheduled** on a different node.
- Hadoop divides the input into fixed-size pieces called input splits (or just splits). It creates **one map task for each split**, which runs the map function for each record in the split.
- **Why many small splits?** The time to process each split is small compared to the whole input → work is **better load-balanced**: a faster machine can process proportionally more splits than a slower one.

Data locality optimisation

Hadoop does its best to run the map task on a node where the input data resides in HDFS, because it doesn't use valuable cluster bandwidth. This is called the **data locality optimisation**.

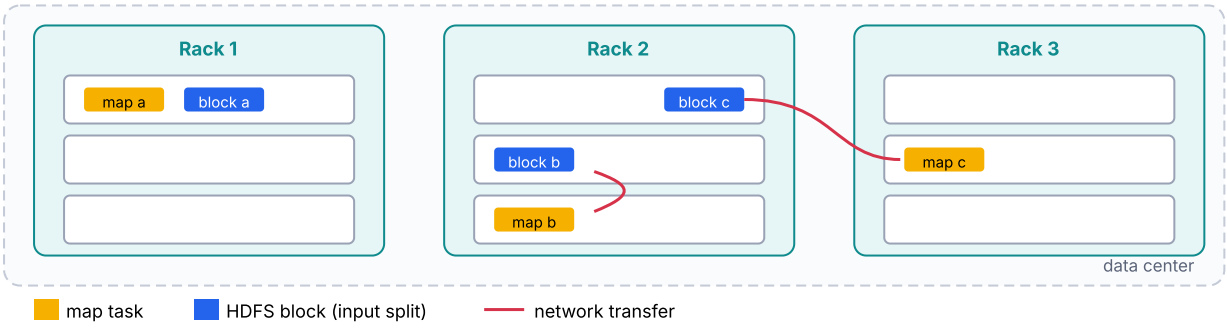


Figure 1.9 — (a) data-local, (b) rack-local and (c) off-rack map tasks (Textbook Fig. 2-2)

Case	When it happens	Cost
(a) Data-local	Map task runs on the **same node** that stores the block — the preferred case.	✅ No network transfer
(b) Rack-local	All nodes holding the block's replicas are busy → scheduler finds a free map slot on a node **in the same rack**.	⚠️ Transfer within the rack
(c) Off-rack	Very occasionally even that is impossible → a node **in a different rack** is used.	❌ Inter-rack network transfer

🔑 **WHY IS THE OPTIMAL SPLIT SIZE = HDFS BLOCK SIZE (128 MB)?**

Because a block is the largest size of input that can be guaranteed to be stored on a single node. If a split spanned two blocks, it's unlikely any one node stores both, so part of the split would have to travel over the network — less efficient than running the whole map task on local data.

🧠 **MEMORY TRICK**

"**Home → Neighbour → Other town**": data-local (your own house), rack-local (your neighbour on the same street), off-rack (another town — slowest trip).

Data flow with a single reduce task

- Map tasks write their output to the **local disk**, not to HDFS — map output is **intermediate**: it's processed by reducers and thrown away once the job is complete.
- The reduce task's input is the **sorted map outputs** from all mappers, transferred over the network (**copy**), **merged**, then passed to the reduce function.
- Reduce output is stored in **HDFS** for reliability (first replica on the local node, other replicas off-rack).

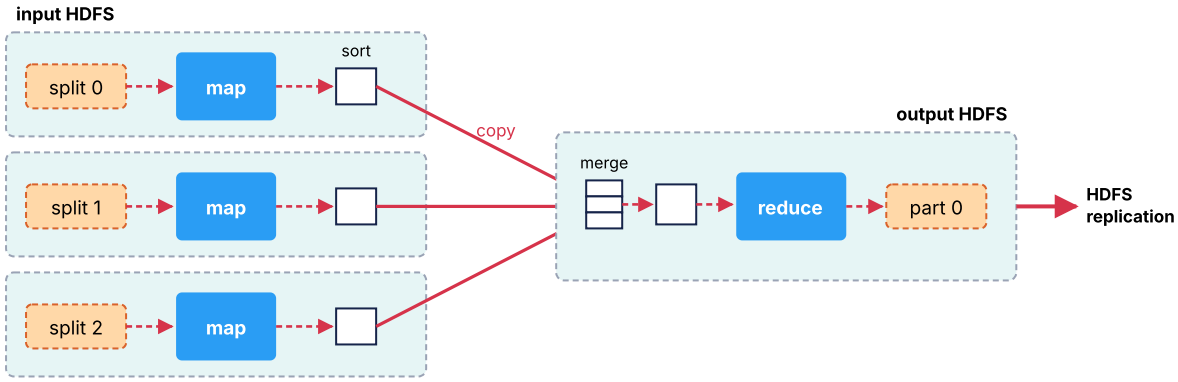


Figure 1.10 — MapReduce data flow with a single reduce task (Textbook Fig. 2-3). Dotted boxes = nodes; dotted arrows = transfer on a node; solid arrows = transfer between nodes.

Data flow with multiple reduce tasks

- The **number of reduce tasks is not governed by the size of the input** — it is specified independently.
- With multiple reducers, map tasks partition their output, creating **one partition for each reduce task**.
- There can be many keys in each partition, but **all records for a given key are in a single partition**.
- Partitioning can be controlled by a **user-defined partitioning function**, but normally the **default partitioner — which buckets keys using a hash function** — works very well.
- Because each reduce task is fed by many map tasks, this data flow is colloquially known as "the shuffle".

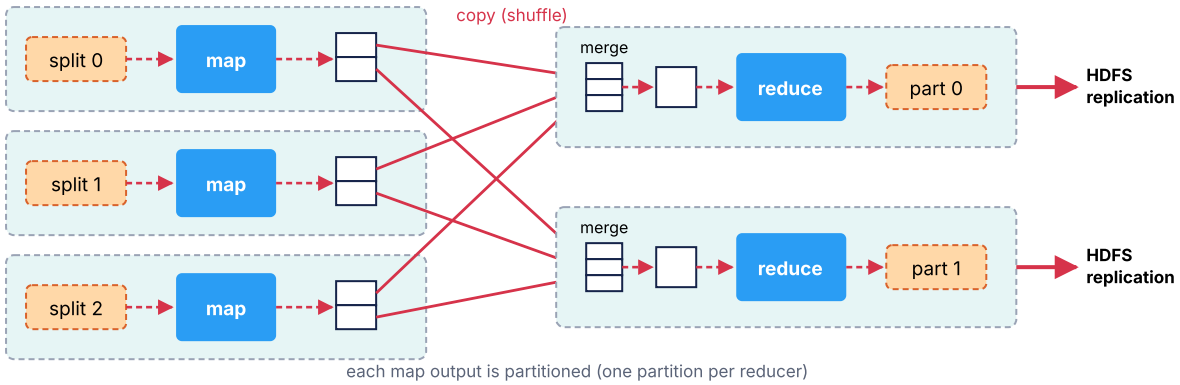


Figure 1.11 — MapReduce data flow with multiple reduce tasks (Textbook Fig. 2-4)

💡 EXTRA (TEXTBOOK): ZERO REDUCERS & COMBINERS

A job can have **zero reduce tasks** when no shuffle is needed — then map tasks write output straight to HDFS. A **combiner** is an optional "mini-reducer" run on map output to cut the data sent over the network (e.g., max of a map's own values first). Your slide's driver uses `job.setCombinerClass(MaxTemperatureReducer.class)` for this.

✅ Before you move on — can you...

- ☐ Quote the data-scale examples (NYSE, Facebook, LHC) and the 0.18 → 1.8 ZB growth?
- ☐ Explain why bigger disks didn't help (1990 vs 20 years later) and the 100-disk idea?
- ☐ Name the two problems of parallel disks and their solutions (replication, MapReduce)?
- ☐ Define Hadoop, list its creators, core components, history and why to use it?
- ☐ Describe the NCDC record format and why -0128 means -12.8 °C?
- ☐ Trace Map → Shuffle & Sort → Reduce for the max-temperature example (Fig 1.8)?
- ☐ Explain the Mapper's four generic types and the job of the Driver?
- ☐ Define job, task, split; explain data-local/rack-local/off-rack and why split = block size?
- ☐ Draw single- and multiple-reducer data flows and explain partitioning?

CHAPTER 2

The Hadoop Distributed File System

What you'll learn

1. What a distributed file system is
2. Design of HDFS — and where it fails
3. Blocks (128 MB) & their benefits
4. NameNode, DataNode, Secondary NameNode
5. HDFS Federation
6. HDFS High Availability
7. Anatomy of a file read (6 steps)
8. Anatomy of a file write (7 steps)

⏱ ≈ 2 hours | Slides: Lecture 2 (pp. 67–88) | Tom White Ch. 3

2.1 Distributed file systems

IN SIMPLE WORDS

When a file is too big for one computer, we **cut it into pieces and store the pieces on many computers** connected by a network. Software that manages files spread over many machines like this is called a **distributed file system**. Hadoop's one is called **HDFS**.

- When a dataset **outgrows the storage capacity of a single physical machine**, it becomes necessary to **partition it across a number of separate machines**.
- File systems that manage the storage across a network of machines are called distributed file systems (DFS). A DFS allows access to files from **multiple hosts sharing via a computer network**.
- Being network-based, they are **more complex** than regular disk file systems.
- One of the biggest challenges: making the file system tolerate node failure without suffering data loss.
- Hadoop's DFS is **HDFS**. Hadoop can also work with other storage (the **local file system**, **Amazon S3**).

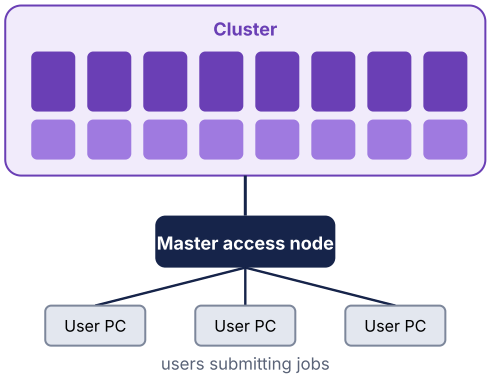


Figure 2.1 — A distributed file system

2.2 The design of HDFS

DEFINITION — LEARN WORD-FOR-WORD

HDFS is a file system designed for storing very large files with streaming data access patterns, running on clusters of commodity hardware.

📦 Very large files

Files that are **hundreds of MB, GB or TB** in size. Hadoop clusters today store **petabytes** of data.

🌊 Streaming data access

Built around write-once, read-many-times. A dataset is generated/copied once, then analysed many times. Each analysis reads a large part (or all) of the dataset, so the **time to read the whole dataset** matters more than the **latency of reading the first record**.

🖥 Commodity hardware

Doesn't need expensive, highly reliable hardware — runs on **commonly available hardware** from multiple vendors. Node failure is likely in large clusters, so HDFS is designed to **keep working without noticeable interruption**.

🧠 **MEMORY TRICK**

HDFS is good at "**L S C**" → **L**arge files, **S**treaming access, **C**heap (commodity) hardware.

Where HDFS does NOT work well

❌ 1. Low-latency data access

Not suitable for apps needing access in the **tens of milliseconds**. HDFS is optimised for **high data throughput**, which may come at the expense of latency. → Use HBase instead for low-latency access.

❌ 2. Lots of small files

The **NameNode holds file-system metadata in memory**; every file, directory and block uses memory. So the number of files is limited by NameNode memory: **millions** of files are feasible, **billions** are beyond current hardware.

Rule of thumb (textbook): each file/dir/block ≈ 150 bytes of NameNode memory.

❌ 3. Multiple writers, arbitrary file modifications

A file is written by a **single writer**. Writes are always made **at the end of the file** (append-only). **No** multiple writers, **no** modifications at arbitrary offsets.

🧠 **MEMORY TRICK — HDFS WEAKNESSES**

"**L S M**": HDFS is **not** for **L**ow latency, **S**mall files, or **M**ultiple writers. (Opposite of its strengths: large files, streaming, one writer.)

2.3 HDFS concepts — Blocks

- A **disk** has a block size — the **minimum amount of data it can read or write**. Disk blocks are normally **512 bytes**.
- File systems for a single disk deal with data in blocks that are an integral multiple of the disk block size — typically **a few kilobytes**.
- This is usually transparent to the user, but tools like `df` and `fsck` ("file system check") work at block level.
- HDFS also has blocks, but a much larger unit — 128 MB by default. Files in HDFS are broken into **block-sized chunks**, stored as **independent units**.
- Unlike a single-disk file system, a file smaller than a block does not occupy a full block of storage — e.g., a **1 MB file** with a 128 MB block size uses **1 MB** of disk, not 128 MB.

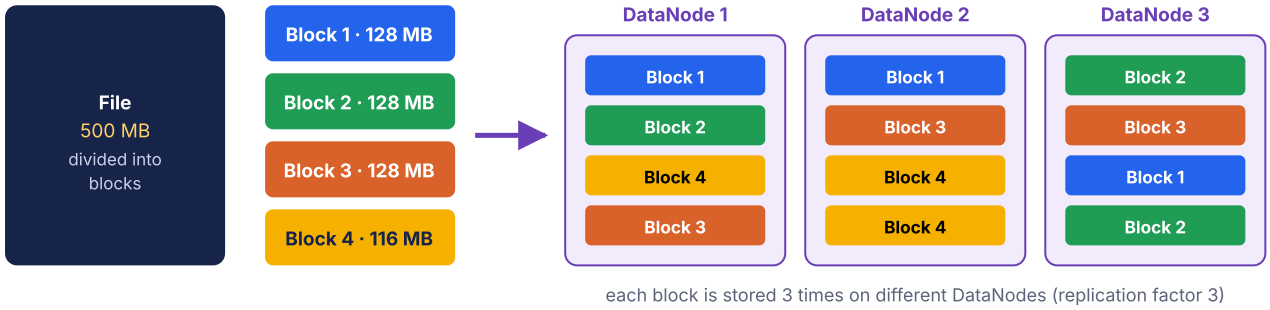


Figure 2.2 — A 500 MB file split into 128 MB blocks, each replicated 3 times across DataNodes. Note the last block is only 116 MB.

Benefits of the block abstraction

1 · File bigger than any disk

A file can be **larger than any single disk** in the network — blocks don't need to be on the same disk, so they can use any disk in the cluster.

2 · Simpler storage subsystem

Making the unit a **block rather than a file** simplifies storage management (fixed size → easy to calculate how many fit on a disk; metadata handled separately). Simplicity matters in distributed systems where failure modes are varied.

3 · Fits replication

Blocks fit well with **replication** for fault tolerance and availability: each block is replicated to a small number of **physically separate machines (typically three)**, guarding against corrupted blocks and disk/machine failure.

⚠ **COMMON MISTAKE**

Older textbook editions/slides may say **64 MB** (Hadoop 1). Your slides say 128 MB by default (Hadoop 2+) — write 128 MB. Also: HDFS block (128 MB) ≫ normal file system block (few KB) ≫ disk block (512 bytes).

2.4 NameNodes and DataNodes

🗣 IN SIMPLE WORDS

Think of a **library**. The **NameNode** is the librarian's **catalogue**: it knows which book (file) is made of which pages (blocks) and which shelf (DataNode) holds each page — but it doesn't hold the pages itself. The **DataNodes** are the **shelves** that actually store the pages.

An HDFS cluster has two types of nodes operating in a master–worker (master–slave) pattern: a **NameNode** (the master) and a number of **DataNodes** (workers).

👑 NameNode (master)

- Manages the file system namespace.
- Maintains the **file system tree** and the **metadata** for all files and directories.
- Stored persistently on local disk as two files: the namespace image and the edit log.
- Knows which DataNodes hold the blocks of each file (this block-location map is rebuilt from DataNode reports at startup, kept in memory).

👷 DataNodes (workers)

- The workhorses of the file system.
- **Store and retrieve blocks** when told to (by clients or the NameNode).
- **Report back to the NameNode periodically** with lists of blocks they are storing (block reports / heartbeats).

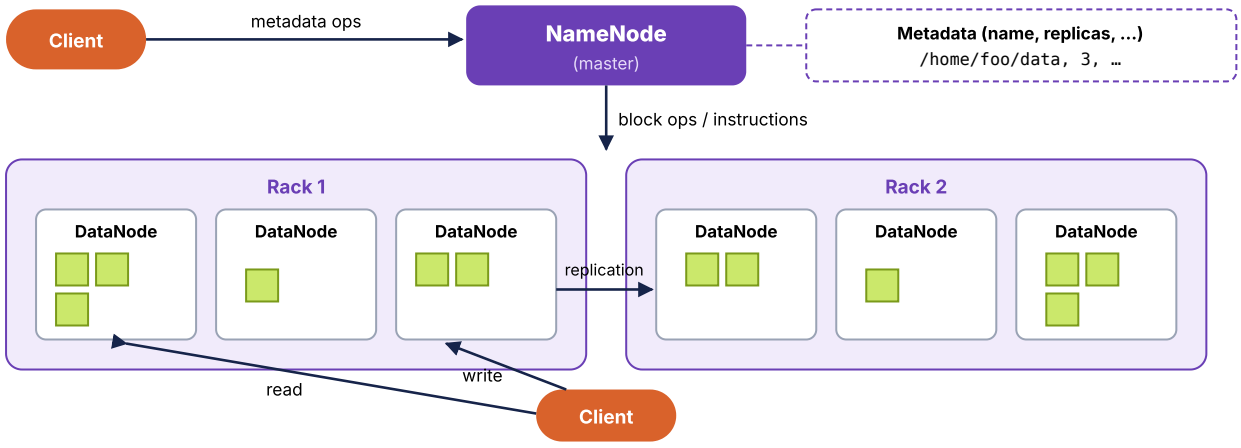


Figure 2.3 — HDFS architecture: one NameNode holds metadata; DataNodes in racks hold replicated blocks; clients read/write blocks directly from/to DataNodes

🔑 KEY POINT

Without the NameNode, the file system cannot be used. If the machine running the NameNode were destroyed, **all files would be lost** — there'd be no way to know how to rebuild files from the blocks on the DataNodes. So the NameNode must be made **resilient to failure**.

Making the NameNode resilient — two mechanisms

1 · Back up the metadata files

Back up the files that make up the persistent state of the file-system metadata. Hadoop can be configured so the NameNode writes its persistent state to **multiple file systems**. These writes are **synchronous and atomic**. Usual choice: write to **local disk + a remote NFS mount** (Network File System).

2 · Run a Secondary NameNode

Despite its name, it does not act as a NameNode. Its main role is to **periodically merge the namespace image with the edit log** to stop the edit log becoming too large. Runs on a **separate machine** (needs lots of CPU and as much memory as the NameNode). Keeps a copy of the merged namespace image, usable if the NameNode fails.

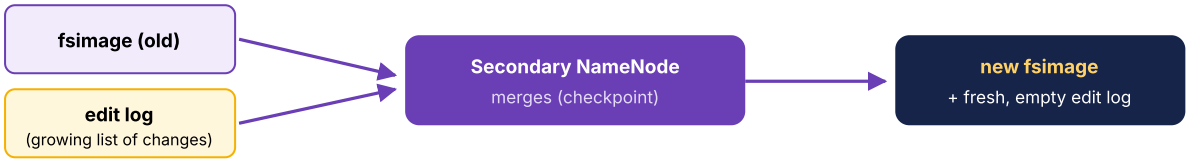


Figure 2.4 — The Secondary NameNode creates checkpoints by merging the namespace image (fsimage) with the edit log

⚠️ **COMMON MISTAKE**

The **Secondary NameNode is NOT a backup/standby NameNode**. It only does **checkpointing** (merging). If the NameNode dies, the Secondary can't take over automatically — that's what the **Standby NameNode** in HDFS High Availability (§2.6) is for.

Working of HDFS (summary)

File is divided into blocks (Block 1...4) → **NameNode** decides which DataNodes store which block → **DataNodes** store the blocks on their local disks (HDFS) → **Replicas** copied between DataNodes for safety

2.5 HDFS Federation

🗣 **IN SIMPLE WORDS**

One librarian can only remember so many books. In a huge library, **hire more librarians and give each one a section** (e.g., one handles `/user`, another handles `/share`). That's federation.

- The NameNode keeps a reference to **every file and block in memory** → on very large clusters with many files, memory becomes the limiting factor for scaling.
- **HDFS Federation**, introduced in the **2.x release series**, allows a cluster to scale by adding NameNodes, each of which manages a portion of the file system namespace.
- Example: one NameNode manages all files under `/user`, a second handles files under `/share`.
- Each NameNode manages a **namespace volume** = the **metadata for the namespace** + a **block pool** containing all the blocks for the files in that namespace.

- Namespace volumes are **independent**: NameNodes **do not communicate** with one another, and the failure of one NameNode does not affect the availability of namespaces managed by others.

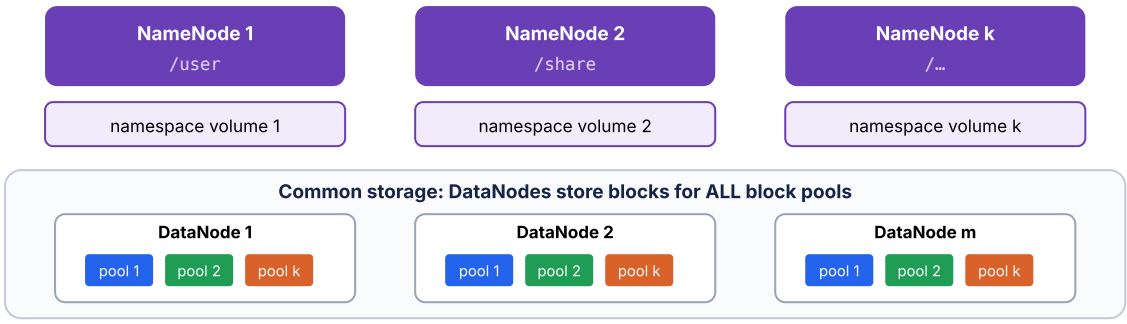


Figure 2.5 — HDFS Federation: several independent NameNodes, each with its own namespace volume (metadata + block pool)

2.6 HDFS High Availability (HA)

The problem: the NameNode is a single point of failure

- Backing up metadata to multiple file systems + a Secondary NameNode protects against **data loss**, but does not provide high availability.
- The NameNode is still a single point of failure (SPOF). If it fails, **all clients — including MapReduce jobs — can't read, write, or list files**, because the NameNode is the sole repository of metadata and the file-to-block mapping. The whole Hadoop system is out of service until a new NameNode is online.
- **Manual recovery:** an administrator starts a new primary NameNode with one of the metadata replicas and configures DataNodes and clients to use it. The new NameNode can't serve requests until it has:
  1. (i) **loaded its namespace image** into memory,
  2. (ii) **replayed its edit log**, and
  3. (iii) **received enough block reports** from DataNodes to leave **safe mode**.

  On large clusters this "cold start" can take 30 minutes or more.

The solution: an active–standby pair of NameNodes

Hadoop 2 adds HA with a pair of NameNodes in an **active–standby** configuration. If the **active** NameNode fails, the **standby** takes over its duties and continues servicing client requests **without a significant interruption**. Architectural changes needed:

1 · Shared edit log

NameNodes use **highly available shared storage** to share the edit log. When the standby comes up, it reads up to the end of the shared edit log to sync with the active one, then keeps reading new entries as the active writes them.

2 · Block reports to both

DataNodes must send **block reports to both NameNodes**, because block mappings are stored in a NameNode's **memory**, not on disk.

3 · Client failover

Clients must be configured to **handle NameNode failover**, using a mechanism that is **transparent to users**.

4 · Standby replaces Secondary

The Secondary NameNode's role is **subsumed by the standby**, which takes **periodic checkpoints** of the active NameNode's namespace.

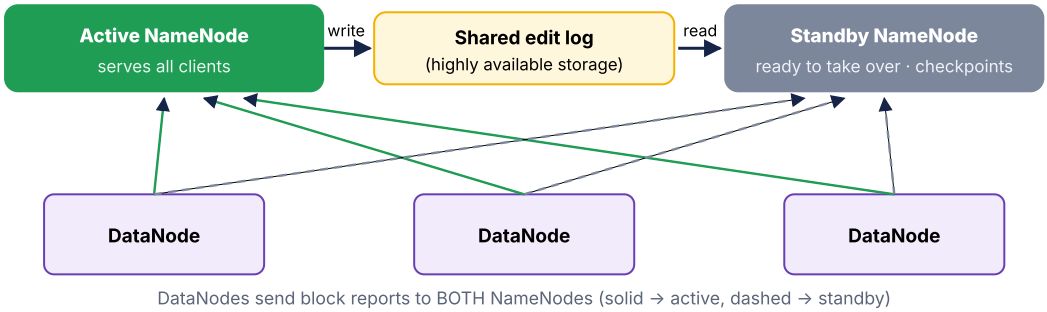


Figure 2.6 — HDFS High Availability with an active–standby NameNode pair

🎯 EXAM TIP

Typical question: "Explain HDFS Federation and High Availability." Federation solves **scaling** (memory limit → more NameNodes, each owning part of the namespace). HA solves the **single point of failure** (standby NameNode takes over). Write **problem → solution → diagram** for each.

2.7 Data flow — Anatomy of a file read

🗣 IN SIMPLE WORDS

To read a file, the client first **asks the NameNode "where are the blocks?"**, then **reads each block directly from the nearest DataNode**, one after another. The NameNode only gives directions — the actual data never passes through it.

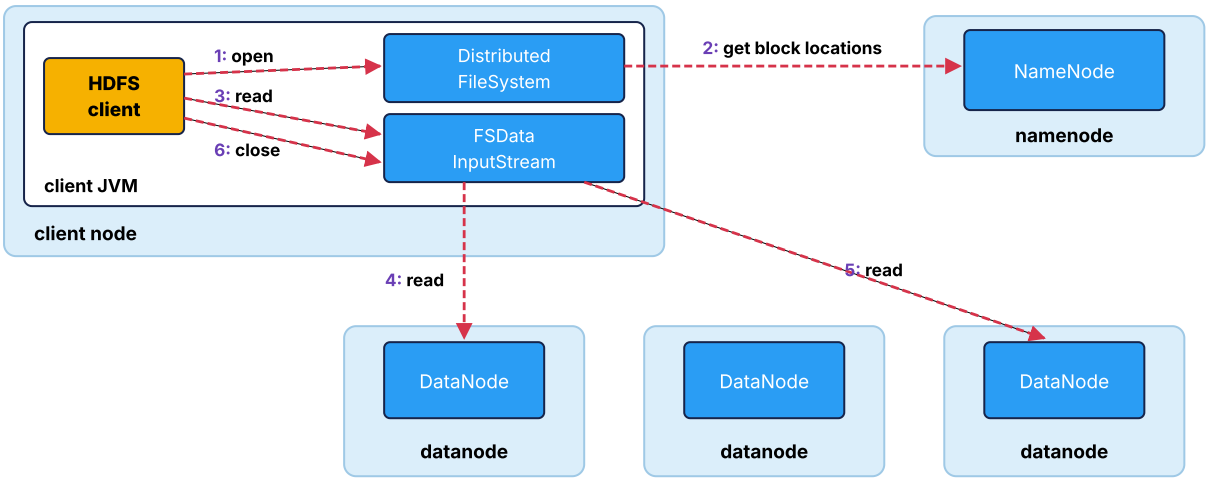


Figure 2.7 — A client reading data from HDFS (Textbook Fig. 3-2). Follow steps 1 → 6.

1. The client **opens** the file by calling `open()` on the `FileSystem` object, which for HDFS is an instance of **DistributedFileSystem**.

2. DistributedFileSystem calls the **NameNode**, using **remote procedure calls (RPCs)**, to get the **locations of the first few blocks** of the file. For each block, the NameNode returns the addresses of the DataNodes that have a copy, sorted by proximity to the client (based on the cluster's network topology). DistributedFileSystem returns an **FSDataInputStream** (supports file seeks) to the client, which wraps a **DFSInputStream** that manages DataNode and NameNode I/O.

3. The client calls `read()` on the stream. DFSInputStream, which has stored the DataNode addresses for the first few blocks, **connects to the first (closest) DataNode** for the first block.

4. Data is **streamed from the DataNode back to the client**, which calls `read()` repeatedly on the stream.

5. When the end of the block is reached, DFSInputStream **closes the connection** to that DataNode, then **finds the best DataNode for the next block**. Blocks are read **in order**; DFSInputStream also asks the NameNode for the locations of the **next batch of blocks** as needed.

6. When the client has finished reading, it calls `close()` on the FSDataInputStream.

🔧 If a DataNode fails during reading

DFSInputStream **tries the next closest DataNode** for that block. It also **remembers failed DataNodes** so it doesn't needlessly retry them for later blocks.

🧪 If a block is corrupted

DFSInputStream **verifies checksums** for data transferred from DataNodes. If a corrupted block is found, it **reads a replica from another DataNode** and **reports the corrupted block to the NameNode**.

🔑 WHY THIS DESIGN IS GOOD

The client **contacts DataNodes directly** to get data, and the NameNode only answers **block-location requests** (held in memory → very fast). This lets HDFS **scale to many concurrent clients**, because data traffic is spread across all DataNodes and the NameNode doesn't become a bottleneck.

🧠 MEMORY TRICK — FILE READ (6 STEPS)

"Open · Get · Read · Read · Read-next · Close" → **O G R R R C** — "**O**nly **G**ood **R**eaders **R**eally **R**ead **C**arefully".

2.8 Data flow — Anatomy of a file write

🗣 IN SIMPLE WORDS

To write a file, the client asks the NameNode to **create** it, then sends data in small **packets** down a **chain (pipeline) of 3 DataNodes**: DataNode 1 saves and passes it to DataNode 2, which saves and passes it to DataNode 3. **Acknowledgements** flow back up the chain.

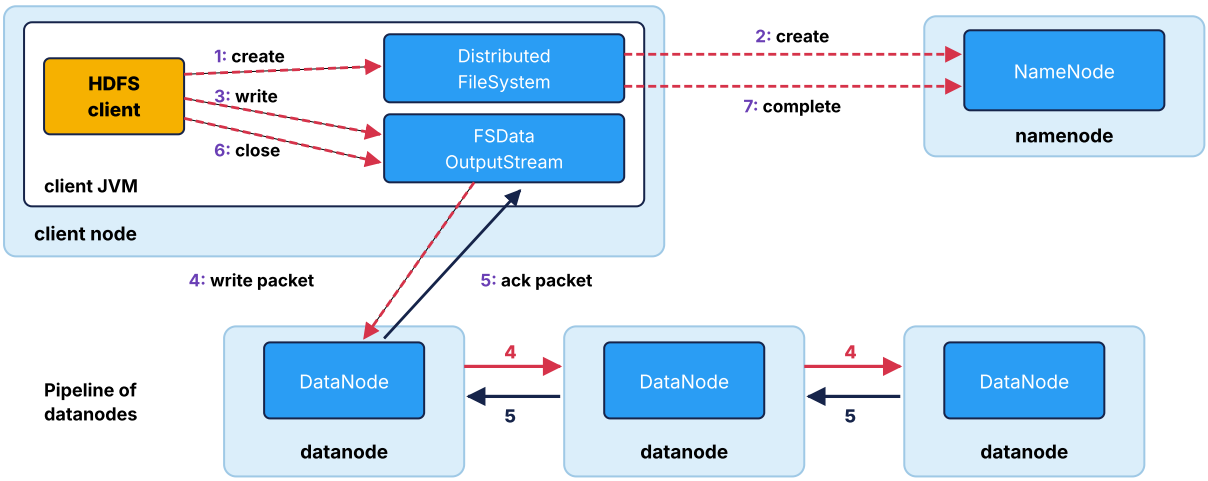


Figure 2.8 — A client writing data to HDFS (Textbook Fig. 3-4). Follow steps 1 → 7. Red = data packets; dark = acknowledgements.

1 The client **creates** the file by calling `create()` on **DistributedFileSystem**.

2 DistributedFileSystem makes an **RPC call to the NameNode** to create a new file in the namespace, **with no blocks associated with it**. The NameNode performs **checks**: the file **doesn't already exist** and the client has the **right permissions**. If checks pass → it records the new file; otherwise file creation fails and the client gets an **IOException**. DistributedFileSystem returns an **FSDataOutputStream** (wrapping a **DFSOutputStream**).

3 As the client **writes** data, DFSOutputStream splits it into packets, written to an internal queue called the data queue. The data queue is consumed by the **DataStreamer**, which asks the NameNode to **allocate new blocks** by picking a list of suitable DataNodes to store the replicas. The list of DataNodes forms a pipeline — with replication level 3, there are **three nodes** in the pipeline.

4 The DataStreamer streams packets to the **first DataNode**, which stores each packet and **forwards it to the second**; the second stores it and forwards it to the **third (last)** DataNode.

5 DFSOutputStream also keeps an internal queue of packets waiting to be acknowledged, called the ack queue. A packet is removed from the ack queue **only when it has been acknowledged by all DataNodes in the pipeline**.

6 When the client has finished writing, it calls `close()` on the stream. This **flushes all remaining packets** to the pipeline and **waits for acknowledgements**...

7 ...before contacting the NameNode to signal that the **file is complete**.

🔧 If a DataNode fails while data is being written

1. The **pipeline is closed**, and any packets in the **ack queue are added to the front of the data queue**, so DataNodes downstream of the failed node don't miss any packets.
2. The current block on the **good DataNodes** is given a **new identity**, communicated to the NameNode, so the **partial block on the failed DataNode will be deleted** if it recovers later.
3. The **failed DataNode is removed** from the pipeline, and a **new pipeline is built from the two good DataNodes**. The rest of the block is written to them. (The NameNode notices the block is under-replicated and arranges a further replica on another node.)

	File read	File write
Starts with	`open()` on DistributedFileSystem	`create()` on DistributedFileSystem
NameNode's job	Return block locations (sorted by closeness)	Check (exists? permission?), record file, allocate blocks/DataNodes, mark complete
Stream class	FSDataInputStream → DFSInputStream	FSDataOutputStream → DFSOutputStream (+ DataStreamer)
Data path	From the **closest** DataNode, block by block	**Pipeline** of 3 DataNodes, packet by packet
Queues	—	**Data queue** + **ack queue**
Failure handling	Try next closest DataNode; checksum → read another replica, report corrupt block	Close pipeline, re-queue packets, new block ID, rebuild pipeline with good nodes
Steps	**6**	**7**

🧠 MEMORY TRICK — FILE WRITE (7 STEPS)

"Create · Create · Write · Packet · Ack · Close · Complete" → **"C C W P A C C"**: "**C**ute **C**ats **W**rite **P**oems **A**nd **C**lose **C**arefully". Steps 1 & 2 are both "create", steps 6 & 7 are close & complete.

🎯 EXAM TIP

"Explain the anatomy of a file read / write in HDFS" = 10-mark question. Draw the figure with **numbered arrows**, then write each numbered step in 1–2 lines, then add the **failure handling** paragraph. Mention class names (DistributedFileSystem, FSDataInputStream/OutputStream, DataStreamer) for full marks.

✅ **Before you move on — can you...**

☐ Define a distributed file system and state the HDFS design definition word-for-word?

☐ Explain the 3 design goals (large files, streaming, commodity) and 3 weak spots (latency, small files, multiple writers)?

☐ Explain HDFS blocks (128 MB), the 1 MB-file example and the 3 benefits of blocks?

☐ Explain NameNode vs DataNode, namespace image & edit log, and the 2 resilience mechanisms?

☐ Explain the Secondary NameNode and why it is NOT a backup?

☐ Explain Federation (namespace volume, block pool) and HA (SPOF, 30-min cold start, 4 changes)?

☐ Draw and explain the file read (6 steps) and file write (7 steps) including failure handling?

CHAPTER 3

Developing a MapReduce Application

What you'll learn

1. MapReduce workflows & JobControl
2. Unit testing with MRUnit
3. How a job is launched (local / classic / yarn)
4. Classic MapReduce (MR1) job run
5. YARN (MR2) job run
6. Failures in MR1 and YARN
7. FIFO, Fair & Capacity schedulers
8. Shuffle and sort

⏱ ≈ 3.5 hours (two sessions) | Slides: Lecture 3 (pp. 89–129) | Tom White Ch. 5–7

3.1 MapReduce workflows

🗣 **IN SIMPLE WORDS**

Finding the max temperature needed only **one** MapReduce job. Harder problems are solved by **chaining several simple jobs** — the output of one job becomes the input of the next — instead of writing one very complicated job.

- When processing gets more complex, the complexity is generally shown by having more MapReduce jobs, rather than more complex map and reduce functions.
- **Rule of thumb:** think about **adding more jobs**, rather than adding complexity to jobs.
- For more complex problems, consider a **higher-level language** than MapReduce: Pig Hive Cascading Crunch Spark. Benefit: they **translate your work into MapReduce jobs for you**, so you can concentrate on the analysis.

Decomposing a problem into MapReduce jobs

📌 **GOAL**

Find the **mean maximum temperature** for every **station + day/month** across many years. E.g., for station `029070-99999` on **January 1**: take the max temperature on Jan 1 1901, Jan 1 1902, … Jan 1 2000, then take the **mean** of those maxima.

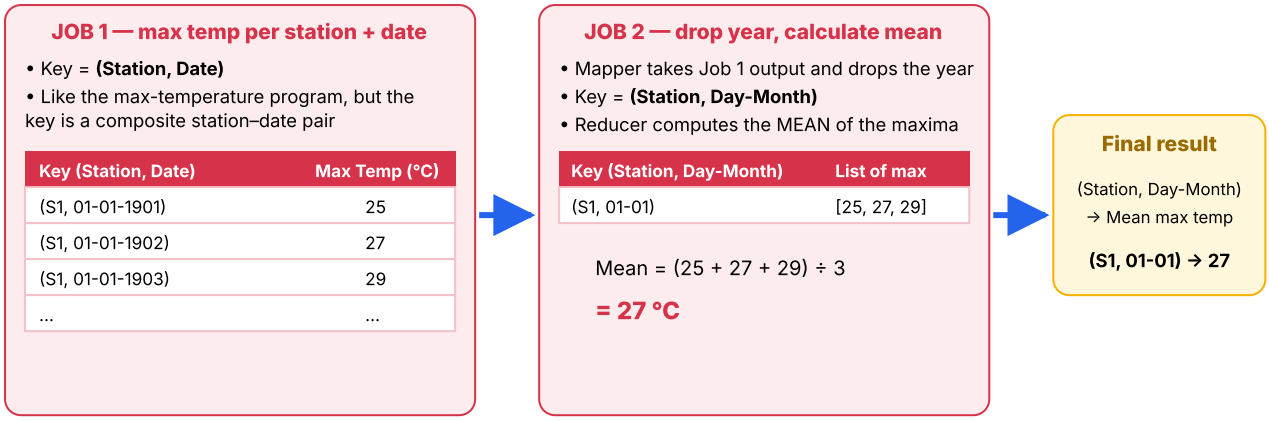
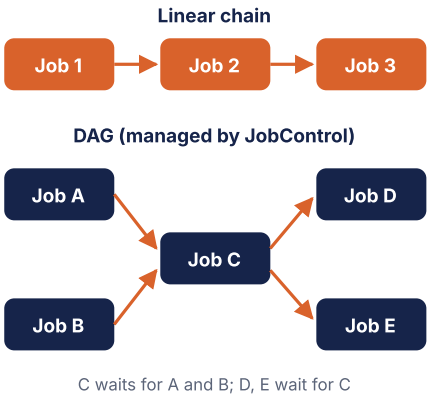


Figure 3.1 — Key idea: complex problem → break into multiple MapReduce jobs

JobControl — running jobs in the right order

- With multiple MapReduce jobs, they must be **executed in the correct order**.
- If jobs form a **linear chain** (Job 1 → Job 2 → Job 3), run them one after another:
  `JobClient.runJob(conf1)` runs the first job;
  `JobClient.runJob(conf2)` runs the second **only after the first finishes successfully**. If a job fails, `runJob()` throws an **IOException**, so later jobs are not executed.
- For more complex workflows, jobs may form a DAG (Directed Acyclic Graph) with different dependencies. JobControl is used to manage such workflows. It:
  - stores the jobs,
  - defines dependencies between jobs,
  - runs jobs in the correct dependency order,
  - monitors job progress,
  - provides job status and error information,
  - **does not run dependent jobs if a required job fails**.



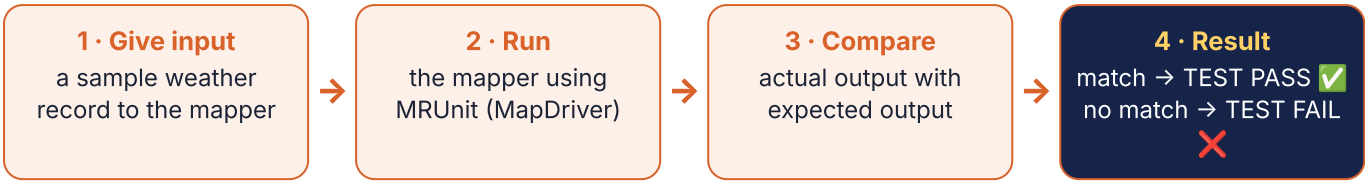
3.2 Writing a unit test with MRUnit

IN SIMPLE WORDS

Before running on a real cluster, we **test** the mapper and reducer on their own: give them a **known input**, and check they produce the **expected output**. MRUnit is the tool that does this.

- Map and reduce functions are **easy to test in isolation** — a consequence of their **functional style**.
- MRUnit is a **testing library** that makes it easy to pass known inputs to a mapper or reducer and check that the outputs are as expected.
- MRUnit is used with a standard test framework such as JUnit, so you can run tests in your normal development environment.

- If the expected output values are not emitted, **MRUnit fails the test**.



Testing the Mapper — MapDriver

We use MRUnit's **MapDriver**, configured with the mapper under test (`MaxTemperatureMapper`), the input key and value, the expected output key (a `Text` for the year **1950**) and expected output value (an `IntWritable` **−11**, i.e., −1.1 °C), then call `runTest()`.

```
@Test
public void processesValidRecord() throws IOException, InterruptedException {
  Text value = new Text("0043011990999991950051518004+68750+023550FM-12+0382" +
                                // Year ^^^^
      "99999V0203201N00261220001CN9999999N9-00111+99999999999");
                                // Temperature ^^^^^
  new MapDriver<LongWritable, Text, Text, IntWritable>()
    .withMapper(new MaxTemperatureMapper())
    .withInput(new LongWritable(0), value)
    .withOutput(new Text("1950"), new IntWritable(-11))   // expected (year, temp)
    .runTest();
}
```

Testing the Reducer — ReduceDriver

Give the reducer a key (**1950**) and a list of values (**10, 5**); verify it outputs the largest (**10**).

```
@Test
public void returnsMaximumIntegerInValues() throws IOException, InterruptedException {
  new ReduceDriver<Text, IntWritable, Text, IntWritable>()
    .withReducer(new MaxTemperatureReducer())
    .withInput(new Text("1950"),
        Arrays.asList(new IntWritable(10), new IntWritable(5)))
    .withOutput(new Text("1950"), new IntWritable(10))       // max of 10 and 5
    .runTest();
}
```

Running a job in a local job runner (testing the whole job)

Using the **Tool** interface, we write a driver (`MaxTemperatureDriver extends Configured implements Tool`) and test it with a **local, in-process job runner** on a small amount of known data:

```java
@Test
public void test() throws Exception {
  Configuration conf = new Configuration();
  conf.set("fs.defaultFS", "file:///");                 // use the local filesystem
  conf.set("mapreduce.framework.name", "local");       // use the local job runner
  conf.setInt("mapreduce.task.io.sort.mb", 1);
  Path input = new Path("input/ncdc/micro");
  Path output = new Path("output");
  FileSystem fs = FileSystem.getLocal(conf);
  fs.delete(output, true);                                  // delete old output
  MaxTemperatureDriver driver = new MaxTemperatureDriver();
  driver.setConf(conf);
  int exitCode = driver.run(new String[] { input.toString(), output.toString() });
  assertThat(exitCode, is(0));
  checkOutput(conf, output);                                  // compare actual vs expected, line by line
}
```

1 · Mapper test

Test only the mapper using **MapDriver**. Input: weather record (1950, −1.1 °C) → expected `(1950, -11)`. −11 = temperature in tenths of a degree.

2 · Reducer test

Test only the reducer using **ReduceDriver**. Input: key 1950, values [10, 5] → expected `(1950, 10)`.

3 · Complete job test

Run the whole job (mapper + shuffle/sort + reducer) on a **local machine** with a **small input file** using the **local job runner**; check the output file matches.

🧠 **MEMORY TRICK**

"**Map**Driver tests the **Map**per, **Reduce**Driver tests the **Reduce**r, **local** job runner tests the **whole job**." Test small pieces first → then the whole.

3.3 Anatomy of a MapReduce job run

- You can run a MapReduce job with a **single method call**: `submit()` on a **Job** object — or `waitForCompletion()`, which **submits the job if not already submitted, then waits for it to finish**. This one call conceals a great deal of processing behind the scenes.
- In Hadoop releases up to **0.20**, the property `mapred.job.tracker` determines the means of execution. If set to `local` (the default), the **local job runner** is used — runs the whole job in a **single JVM**, designed for testing and small datasets.
- In **Hadoop 0.23.0** a new MapReduce implementation was introduced — **MapReduce 2**, built on a system called **YARN**.
- The framework used for execution is set by the property mapreduce.framework.name, which takes the values:

Value	Framework	Where it runs	Main components
`local`	Local job runner	Single machine (one JVM)	Runs the whole job in a single JVM (for testing / small data)
`classic`	MapReduce 1 ("classic")	Hadoop cluster	**JobTracker + TaskTrackers**
`yarn`	MapReduce 2 (new framework)	Hadoop cluster using YARN	**ResourceManager + NodeManagers + ApplicationMaster**

Your MapReduce program → **job.submit() / waitForCompletion()** → **Hadoop checks** mapreduce.framework.name → **local · classic · yarn**

3.4 Classic MapReduce (MapReduce 1)

A job run in classic MapReduce has **four independent entities**:

💻 Client

Submits the MapReduce job.

🧭 JobTracker

Coordinates the job run. A Java application whose main class is `JobTracker`.

👷 TaskTrackers

Run the tasks the job has been split into. Java apps, main class `TaskTracker`.

🗄 Distributed FS

Normally **HDFS** — used for **sharing job files** between the other entities.

🧠 **MEMORY TRICK — MR1 ENTITIES**

"**C J T D**" → "**C**lient **J**ust **T**alks to **D**FS" — Client, JobTracker, TaskTrackers, Distributed file system.

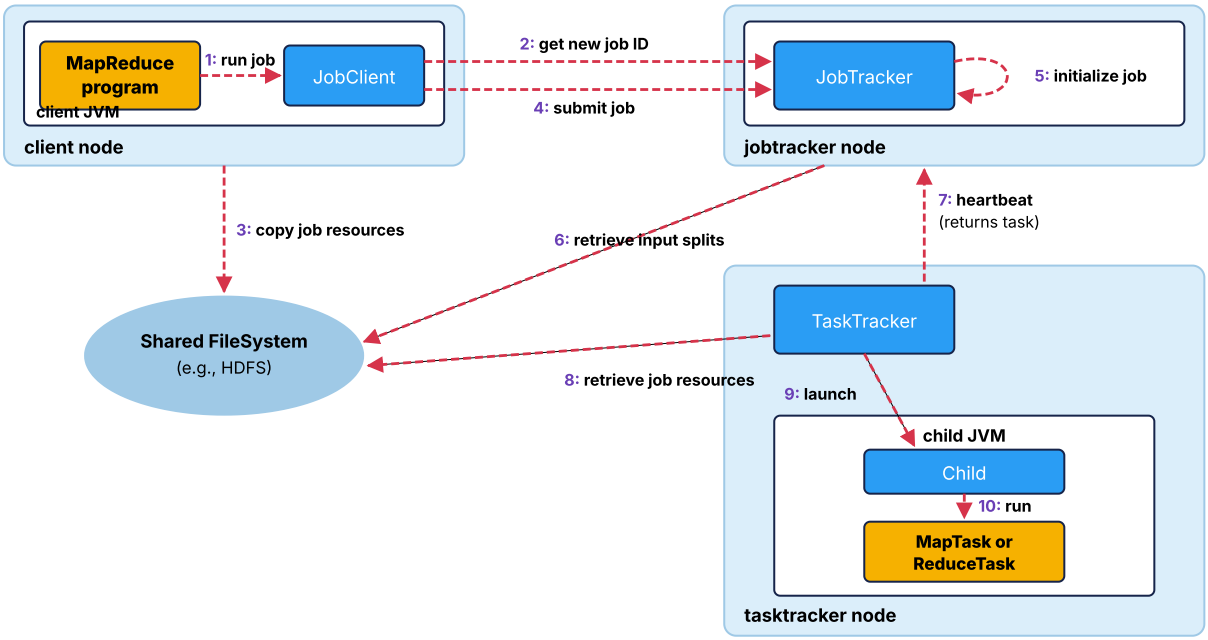


Figure 3.2 — How Hadoop runs a MapReduce job using the classic framework (Textbook Fig. 6-1). Steps 1 → 10.

A · Job submission (steps 1–4)

1 `submit()` on Job creates an internal **JobSubmitter** and calls `submitJobInternal()` on it.

2 Asks the **JobTracker for a new job ID** (`getNewJobId()`). Then it **checks the output specification** — if the output directory isn't specified or **already exists**, the job isn't submitted and an error is thrown. It **computes the input splits** — if they can't be computed (e.g., input paths don't exist), the job isn't submitted.

3 **Copies the resources** needed to run the job — the **job JAR**, the **configuration file** and the **computed input splits** — to the JobTracker's file system (shared FS). The job JAR is copied with a **high replication factor** so there are many copies across the cluster for TaskTrackers to access.

4 Tells the JobTracker the job is **ready for execution** (`submitJob()` on JobTracker).

B · Job initialisation (steps 5–6)

5 The JobTracker puts the job in an **internal queue**; the **job scheduler** picks it up and initialises it — creates an object to represent the job, encapsulating its tasks and **bookkeeping information** to track task status and progress.

6 To create the list of tasks, the scheduler **retrieves the input splits** computed by the client from the shared file system. It creates one map task for each split. The number of reduce tasks is set by the `mapred.reduce.tasks` property (set via `setNumReduceTasks()`). Tasks are given IDs.

C · Task assignment (step 7)

7 TaskTrackers run a simple loop that periodically sends heartbeat calls to the JobTracker. Heartbeats tell the JobTracker that a TaskTracker is **alive**, and also act as a **channel for messages**: the TaskTracker says whether it's ready for a new task, and if so, the JobTracker **allocates a task** via the heartbeat return value.

D · Task execution (steps 8–10)

8 The TaskTracker **localises the job JAR** by copying it from the shared file system to its own file system, and copies any files needed from the **distributed cache** to local disk. It creates a **local working directory** and un-jars the JAR into it.

9 It creates a **TaskRunner**, which **launches a new Java Virtual Machine (JVM)**...

10 ...to **run each task** in — so that bugs in the user's map/reduce functions **don't affect the TaskTracker** (e.g., crash or hang it). (It is possible to reuse the JVM between tasks.)

🧠 **MEMORY TRICK — THE 4 PHASES**

"S I A E" → **S**ubmission · **I**nitialisation · **A**ssignment · **E**xecution — "**S**tudents **I**n **A**n **E**xam". Same 4 phases appear in YARN too!

3.5 YARN (MapReduce 2)

🗣 IN SIMPLE WORDS

In MR1, the **JobTracker did two jobs**: managing cluster resources *and* babysitting every task of every job. On huge clusters it became overloaded. YARN **splits that work**: a **ResourceManager** manages resources for the whole cluster, and each job gets its own **ApplicationMaster** to babysit its tasks.

- For very large clusters — around 4,000 nodes and higher — classic MapReduce hits **scalability bottlenecks**. In **2010** a group at **Yahoo!** began designing the next generation of MapReduce.
- The result was YARN — Yet Another Resource Negotiator (or the recursive acronym *YARN Application Resource Negotiator*).
- YARN fixes the scalability problems by **splitting the responsibilities of the JobTracker** into separate entities. The JobTracker handled both:
  - **job scheduling** — matching tasks with TaskTrackers, and
  - **task progress monitoring** — tracking tasks, restarting failed/slow tasks, bookkeeping such as counter totals.
- YARN separates these into two independent daemons:
  - a ResourceManager to manage the use of resources across the cluster, and
  - an ApplicationMaster to manage the lifecycle of applications running on the cluster.

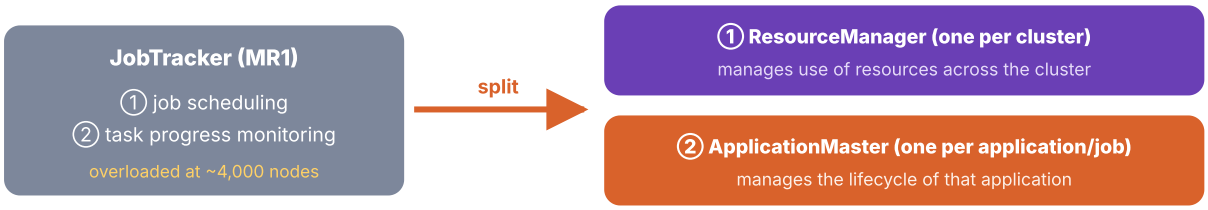


Figure 3.3 — YARN splits the JobTracker's two responsibilities into separate daemons

MapReduce on YARN involves **five independent entities**:

💻 Client
Submits the MapReduce job.

🧭 YARN ResourceManager
Coordinates the **allocation of compute resources** on the cluster.

👷 YARN NodeManagers
Launch and monitor the compute containers on machines in the cluster.

🎬 MapReduce ApplicationMaster
Coordinates the tasks running the MapReduce job. The AM and the tasks run in **containers** scheduled by the ResourceManager and managed by NodeManagers.

🗄 Distributed FS
Normally **HDFS** — used for sharing job files between the other entities.

🧠 Memory trick
"**C R N A D**" → "**C**an **R**ahul **N**ot **A**ttend **D**aily?" — Client, ResourceManager, NodeManagers, ApplicationMaster, DFS.

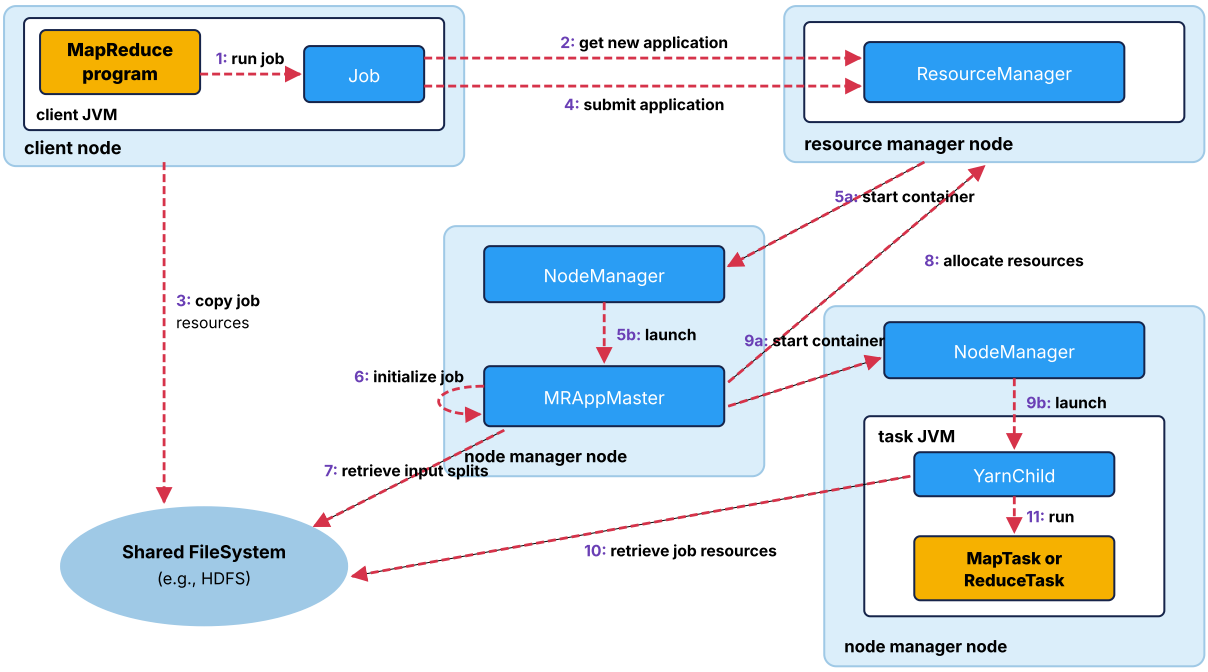


Figure 3.4 — How Hadoop runs a MapReduce job using YARN (Textbook Fig. 7-1). Steps 1 → 11.

A · Job submission (steps 1–4)

1 `submit()` on Job creates an internal **JobSubmitter** and calls `submitJobInternal()`.

2 Asks the **ResourceManager for a new application ID**, used as the MapReduce job ID. **Checks the output specification** (output dir not specified or already exists → error). **Computes the input splits** (can't compute, e.g., input paths don't exist → error).

3 **Copies the resources** needed to run the job — job JAR, configuration file, computed input splits — to the **shared file system** in a directory named after the job ID.

4 Submits the job by calling `submitApplication()` on the ResourceManager.

B · Job initialisation (steps 5–7)

5a 5b The ResourceManager hands the request to the **YARN scheduler**. The scheduler **allocates a container**, and the ResourceManager **launches the ApplicationMaster's process** there, under the NodeManager's management.

6 The ApplicationMaster for MapReduce jobs is a Java application whose main class is MRAppMaster. It **initialises the job** by creating bookkeeping objects to track the job's progress (it will receive progress and completion reports from the tasks).

7 It **retrieves the input splits** computed in the client from the shared file system.

C · Task assignment (step 8)

8 If the job does not qualify to run as an **uber task** (a small job run inside the AM's own JVM), the ApplicationMaster **requests containers for all map and reduce tasks** from the ResourceManager. **Map task requests are made first, with higher priority** than reduce tasks, since all maps must complete before the reduce's sort phase can start. Reduce requests are not made until 5% of map tasks have completed ("reduce slow start").

D · Task execution (steps 9–11)

9a 9b Once a task is assigned a container on a node, the ApplicationMaster **starts the container by contacting the NodeManager**.

10 The task is executed by a Java application whose main class is YarnChild. Before running the task, it **localises the resources** it needs — job configuration, JAR file, and any files from the distributed cache.

11 Finally, it **runs the map or reduce task**.

💡 **PROGRESS & STATUS UPDATES IN YARN (SLIDE FIGURE 7-3)**

Each task (MapTask/ReduceTask inside the **YarnChild**) reports its **progress and counters** to its **MRAppMaster** (status update). The client polls the **MRAppMaster** (`getJobStatus`) to show progress to the user — the ResourceManager is **not** involved in task-level status, which keeps it light.

Classic MapReduce (MR1) vs YARN (MR2)

Point	Classic MapReduce (MR1)	YARN (MR2)
Introduced	Hadoop up to 0.20 / 1.x	Hadoop 0.23 / 2.x (designed at Yahoo!, 2010)
Entities	4: Client, JobTracker, TaskTrackers, DFS	5: Client, ResourceManager, NodeManagers, ApplicationMaster, DFS
Master daemon	**JobTracker** — does scheduling **and** task monitoring	**ResourceManager** (resources) + per-job **ApplicationMaster** (monitoring)
Worker daemon	**TaskTracker** (fixed map/reduce slots)	**NodeManager** (flexible containers)
Task runs in	Child JVM launched by TaskRunner	YarnChild in a container
Job ID from	JobTracker (`getNewJobId()`)	ResourceManager (new application ID)
Scalability	Bottleneck around **4,000 nodes**	Scales much further
Single point of failure	**JobTracker** failure → job fails	Designed to **eliminate SPOFs** (AM restarts, RM recovery)
Steps in job run	10	11 (with 5a/5b, 9a/9b)

🎯 EXAM TIP

"Explain how a MapReduce job runs in classic MapReduce / YARN" = 10 marks. Draw the figure, then write the **4 phases (Submission, Initialisation, Assignment, Execution)** with step numbers. "Compare MR1 and YARN" → use the table above.

3.6 Failures

🗣 IN SIMPLE WORDS

In a cluster of thousands of cheap machines, **something is always breaking**: user code crashes, a machine dies, a network link fails. Hadoop is designed to **notice failures (via missing heartbeats) and retry the work elsewhere**, so the job still finishes.

Failures in Classic MapReduce (MR1) — 3 failure modes

1 · Task failure

- Most common: **user code in the map/reduce task throws a runtime exception**. The child JVM reports the error to its parent **TaskTracker** before it exits; the error ends up in the **user logs**.
- The TaskTracker **marks the task attempt as failed**, freeing a slot for another task.
- **Streaming tasks:** if the Streaming process exits with a **non-zero exit code**, it's marked as failed.
- **Sudden exit of the child JVM** (e.g., a JVM bug): the TaskTracker notices the process has exited and marks the attempt as failed.
- (Textbook: hanging tasks are noticed by a missing progress update after a timeout — default 10 min.)

2 · TaskTracker failure

- If a TaskTracker **crashes or runs very slowly**, it **stops sending heartbeats** to the JobTracker (or sends them very infrequently).
- The JobTracker notices and **removes it from its pool** of TaskTrackers to schedule tasks on.
- Blacklisting: a TaskTracker can be blacklisted even if it hasn't failed. If **more than four tasks from the same job fail** on it, the JobTracker records a **fault**. It's blacklisted if its faults exceed a minimum threshold and are **significantly higher than the cluster average**.

3 · JobTracker failure

- The most serious failure mode.
- Hadoop has **no mechanism** for dealing with it — the JobTracker is a single point of failure — so the **job fails**.
- Low chance of occurring (one particular machine failing is unlikely).
- Good news: improved in **YARN**, one of whose design goals is to **eliminate single points of failure**.

Failures in YARN — 4 entities can fail

What fails	How it's detected	What happens
Task	Runtime exceptions & sudden JVM exits are **propagated back to the ApplicationMaster**. Hanging tasks are noticed by the **absence of a ping over the umbilical channel** (timeout: `mapreduce.task.timeout`).	Task attempt marked as **failed**. Same rules as classic: a task is marked failed after four attempts.
Application Master	The AM sends **periodic heartbeats to the ResourceManager**; the RM detects the failure.	RM **starts a new AM instance in a new container** (managed by a NodeManager). By default an application is marked failed if it fails **once**; increase with `yarn.resourcemanager.am.max-retries`.
Node Manager	Stops sending **heartbeats to the ResourceManager**. RM waits `yarn.resourcemanager.nm.liveness-monitor.expiry-interval-ms` (default **600000 ms = 10 minutes**).	Removed from the RM's pool of available nodes. NodeManagers may be **blacklisted by the ApplicationMaster** if failures are high — the AM reschedules tasks on other nodes if **more than three tasks fail** on a NodeManager.
Resource Manager	—	**Serious**: without it, neither jobs nor task containers can be launched. Designed to **recover from crashes** using **checkpointing** — saving its state to persistent storage. After a crash, a new RM instance is brought up (by an administrator) and **recovers from the saved state** (the NodeManagers and running applications).

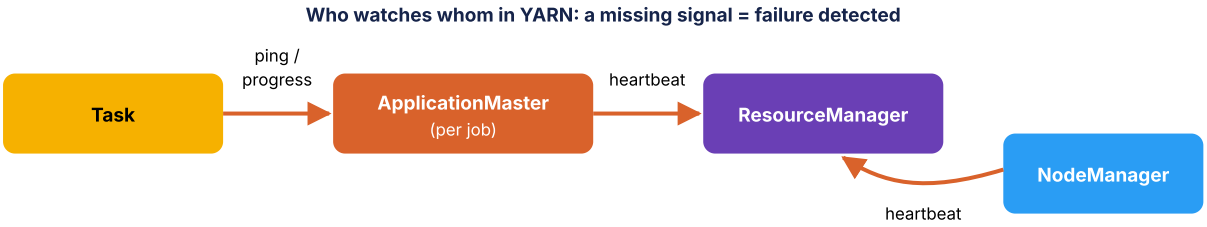


Figure 3.5 — Failure detection in YARN

Failure of...	Classic MR1	YARN
Task	TaskTracker marks attempt failed	ApplicationMaster marks attempt failed (after 4 attempts → task fails)
Worker node	TaskTracker stops heartbeats → removed; blacklisting by JobTracker	NodeManager stops heartbeats → removed after 10 min; blacklisting by AM
Job coordinator	— (JobTracker does this)	ApplicationMaster restarts in a new container
Master	**JobTracker = SPOF → job fails**	ResourceManager recovers from checkpointed state

🧠 **MEMORY TRICK**

MR1 failures: "**T T J**" — **T**ask, **T**askTracker, **J**obTracker ("**T**wo **T**ired **J**ugglers"). YARN failures: "**T A N R**" — **T**ask, **A**pplication master, **N**ode manager, **R**esource manager ("**T**he **A**nts **N**ever **R**est").

3.7 Job scheduling

🗣️ **IN SIMPLE WORDS**

Many users share one cluster. A **scheduler** decides **whose job gets the cluster's resources, and when** — like a teacher deciding the order students use the one computer lab.

MapReduce in Hadoop comes with a choice of schedulers: the default in MapReduce 1 is the original FIFO queue-based scheduler, plus two **multi-user** schedulers: the Fair Scheduler and the Capacity Scheduler.

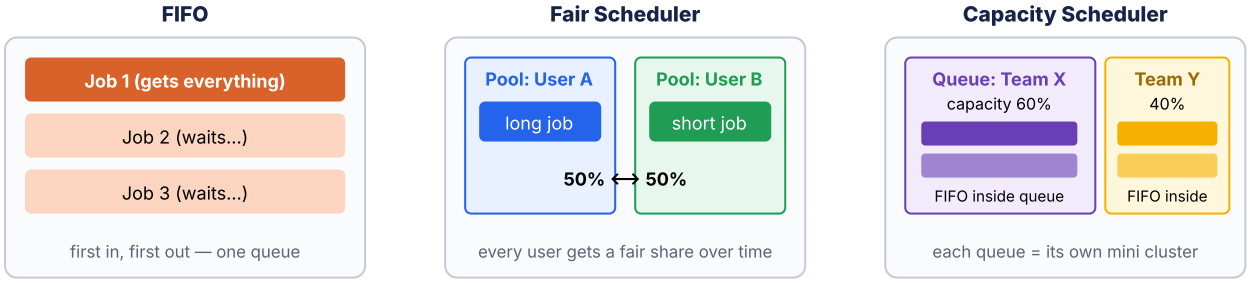


Figure 3.6 — FIFO vs Fair vs Capacity scheduling

⚖️ The Fair Scheduler

- Aims to give every user a fair share of the cluster capacity over time.
- If a **single job** is running, it gets **all of the cluster**.
- As more jobs are submitted, free task slots are given to jobs so each user gets a fair share.
- A **short job** of one user completes in reasonable time even while another user's **long job** runs — and the long job still makes progress.
- Jobs are placed in pools; by default **each user gets their own pool**. A user who submits more jobs than another won't get more cluster resources, on average.
- Supports preemption: if a pool hasn't received its fair share for a certain time, the scheduler **kills tasks in pools running over capacity** to give slots to the pool running under capacity.

📦 The Capacity Scheduler

- A slightly different approach to multi-user scheduling.
- A cluster is made up of a number of queues (like the Fair Scheduler's pools), which may be **hierarchical** (a queue may be the child of another queue). Each queue has an **allocated capacity**.
- Within each queue, jobs are scheduled using **FIFO scheduling (with priorities)**.
- In effect, it allows users or organisations (defined using queues) to simulate a separate MapReduce cluster with FIFO scheduling for each.
- Ideal for **shared clusters** where resource allocation among queues must follow **predefined capacities**.

PREEMPTION

A process in resource scheduling where currently allocated resources are forcefully taken away from one task or job and reassigned to another with higher priority. This ensures critical or high-priority jobs get the resources they need, even if it means interrupting or delaying other tasks.

Feature	Fair Scheduler	Capacity Scheduler
Purpose	Ensures fair resource sharing between users or jobs	Ensures cluster capacity is shared efficiently between multiple queues
Resource allocation	Resources shared equally among jobs/users, allowing fairness	Resources allocated based on queues; each queue gets a configured share
Within a group	Fair sharing within each pool (FIFO optional)	FIFO (with priorities) within each queue
Preemption	Supports preemption — long-running jobs give up resources for higher-priority tasks	Supports preemption, generally used for queue-based preemption
Configuration	Simpler — focuses on fairness & balancing	More complex — set up queues and their capacities
Use case	Multi-user environments that need fair access	Organisations with multiple teams/projects needing strict capacity limits
Flexibility	Dynamic adjustment as jobs complete or arrive	Divided across queues with fixed minimum guarantees
Performance	Balances performance by sharing resources equally	Maximises utilisation by distributing capacity optimally
Fairness concept	Jobs get an equal share of resources over time	Queues get a predefined capacity; unused capacity can be shared

🧠 **MEMORY TRICK**

Fair → **P**ools → "**F**riends **P**lay fair". **C**apacity → **Q**ueues → "**C**ompanies **Q**ueue up, each with its own share". FIFO = "first come, first served".

3.8 Shuffle and sort

DEFINITION

MapReduce guarantees that the input to every reducer is sorted by key. The process by which the system **performs the sort — and transfers the map outputs to the reducers as inputs** — is known as the shuffle.

- The shuffle is an area of the codebase where **refinements and improvements are continually being made**.
- The shuffle is the heart of MapReduce and is where **"the magic" happens**.

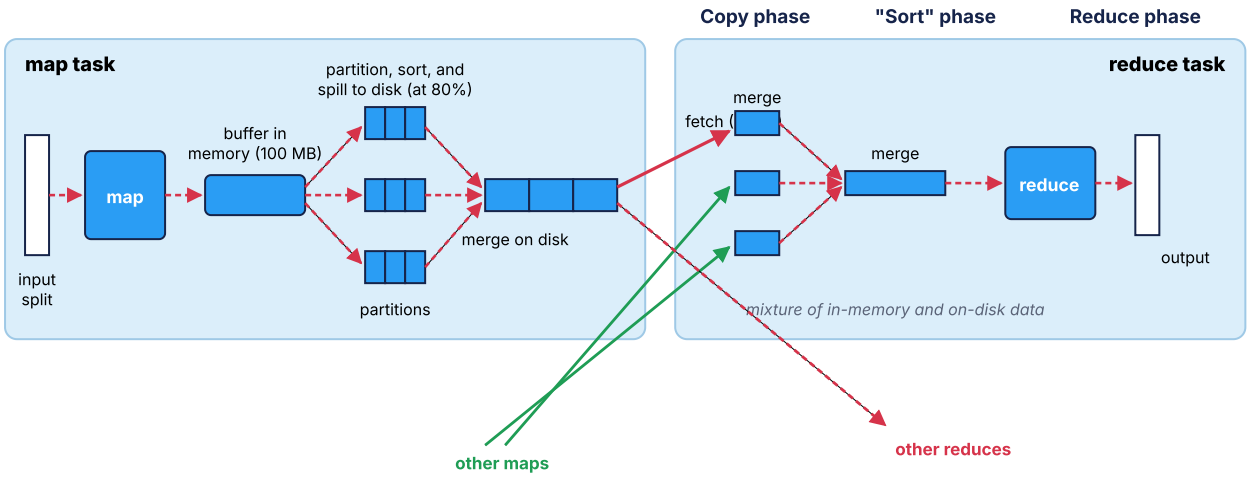


Figure 3.7 — Shuffle and sort in MapReduce (Textbook Fig. 6-6)

The map side

When the map function starts producing output, it is **not simply written to disk**. It uses **buffering writes in memory** and does some **presorting** for efficiency:

1. Each map task has a circular memory buffer that it writes its output to. The buffer is 100 MB by default.

2. When the buffer reaches a threshold (`io.sort.spill.percent`, default 0.80 = 80%), a **background thread starts to spill** the contents to disk. Map outputs continue to be written to the buffer during the spill, but if the buffer **fills up**, the map **blocks until the spill is complete**.

3. Spills are written in **round-robin fashion** to the directories in the `mapred.local.dir` property, in a job-specific subdirectory.

4. Before writing to disk, the thread first divides the data into partitions corresponding to the reducers they'll be sent to.

5. Within each partition, the background thread performs an in-memory sort by key, and if there is a **combiner function**, it is run on the output of the sort. Running the combiner gives a **more compact map output** → less data to write to local disk and to transfer to the reducer.

6. Each time the buffer reaches the spill threshold, a **new spill file** is created — so there could be several spill files. Before the task finishes, the spill files are merged into a single partitioned and sorted output file.

7. It's often a good idea to **compress the map output** as it's written — faster to write, saves disk space, less data to transfer.

8. The output file's partitions are made available to reducers **over HTTP**. The max number of worker threads serving them is set by `tasktracker.http.threads` — **per TaskTracker**, not per map slot.

> 🧠 **MEMORY TRICK — MAP SIDE**
>
> "**B S P S C M**" → **B**uffer (100 MB) → **S**pill at 80% → **P**artition → **S**ort → **C**ombine → **M**erge — "**B**ig **S**pills **P**lease **S**weep, **C**lean, **M**op".

The reduce side

1 The map output file sits on the **local disk of the machine that ran the map task** (map outputs always go to local disk; reduce outputs may not). The reduce task needs the map output for **its particular partition** from **several map tasks** across the cluster.

2 **Copy phase:** map tasks finish at different times, so the reduce task starts copying their outputs as soon as each completes. It uses a small number of **copier threads** to fetch in parallel — default five threads (`mapred.reduce.parallel.copies`).

3 Map outputs are copied into the reduce task JVM's **memory** if small enough; otherwise to **disk**. When the in-memory buffer reaches a threshold size or number of map outputs, it is **merged and spilled to disk** (the combiner runs during this merge if specified). As copies accumulate on disk, a background thread **merges them into larger, sorted files**. Compressed map outputs must be decompressed in memory to merge them.

4 **Sort phase** (really a merge phase — sorting was done on the map side): when all map outputs have been copied, the reduce task merges them **maintaining their sort order**. This is done in **rounds** (e.g., 50 map outputs with merge factor 10 → 5 rounds → 5 intermediate files).

5 Instead of a final round merging those files into one sorted file, the merge **saves a trip to disk** by directly feeding the reduce function in the last phase.

6 **Reduce phase:** the reduce function is invoked for each key in the sorted output. The output is written directly to the output file system — typically **HDFS**.

> 🧠 **MEMORY TRICK — REDUCE SIDE**
>
> "**Copy → Merge (sort) → Reduce**" = "**C M R**" — "**C**ollect, **M**ix, **R**esult". The reducer's "sort" phase is actually a **merge**.

Number to remember	Value	Property
Map-side circular buffer	**100 MB**	`io.sort.mb`
Spill threshold	**0.80 (80%)**	`io.sort.spill.percent`
Reduce-side copier threads	**5**	`mapred.reduce.parallel.copies`
Map output served over	**HTTP**	`tasktracker.http.threads`
HDFS block size	**128 MB**	`dfs.blocksize`
Replication factor	**3**	`dfs.replication`
Task attempts before failing	**4**	`mapreduce.map/reduce.maxattempts`
NodeManager expiry	**10 min (600000 ms)**	`…nm.liveness-monitor.expiry-interval-ms`
Reduce slow start	**5%** of maps done	`mapreduce.job.reduce.slowstart.completedmaps`

🎯 **EXAM TIP**

"Explain shuffle and sort" = 10 marks. Start with the **definition** (input to every reducer is sorted by key; the process of sorting and transferring map outputs = shuffle). Draw **Fig 3.7**, then write **Map side** (buffer, spill, partition, sort, combiner, merge, HTTP) and **Reduce side** (copy, merge/sort, reduce) as numbered points.

✅ **Before you move on — can you…**

- ☐ Explain why complex problems use more jobs, the mean-max example (2 jobs) and JobControl / DAG?
- ☐ Explain MRUnit with MapDriver, ReduceDriver and the local job runner test?
- ☐ List the 3 values of mapreduce.framework.name (local / classic / yarn)?
- ☐ Draw the MR1 job run (10 steps) and explain the 4 phases: submission, initialisation, assignment, execution?
- ☐ Explain why YARN was created and draw the YARN job run (11 steps)?
- ☐ Compare MR1 vs YARN in a table?
- ☐ Explain failures in MR1 (task, TaskTracker, JobTracker) and YARN (task, AM, NM, RM)?
- ☐ Explain FIFO, Fair (pools, preemption) and Capacity (queues) schedulers and compare them?
- ☐ Draw shuffle & sort and explain the map side and reduce side?

LAST-NIGHT REVISION

Final Revision

Use these pages the night before the exam

1. Cheat sheet (all 3 chapters)
2. All memory tricks in one place
3. Important 2-mark questions
4. Important 5/10-mark questions

⏱ ≈ 30–40 minutes

Then redraw the 6 key diagrams once

★ Cheat sheet

CHAPTER 1 · Basics of Hadoop

- **Data!** 2006: 0.18 ZB → 2011: 1.8 ZB (10×). 1 ZB = 10^{21} B = 1 billion TB. NYSE 1 TB/day · Facebook 10 bn photos ≈ 1 PB · Ancestry 2.5 PB · Internet Archive 2 PB (+20 TB/month) · LHC 15 PB/yr. MyLifeBits ≈ 1 GB/month.
- **Storage problem:** 1990 disk 1,370 MB @ 4.4 MB/s (5 min); now 1 TB @ 100 MB/s (> 2.5 hr). Read 1 TB from 100 disks in parallel → < 2 min.
- **2 problems:** hardware failure → **replication** (HDFS); combining data → **MapReduce**.
- **Hadoop** = open-source framework to store & process large data in a parallel, distributed manner; Java; Doug Cutting & Mike Cafarella; = reliable shared storage + reliable distributed analysis.
- **Components:** HDFS (storage), MapReduce (processing), YARN (resource mgmt), Common.
- **History:** Nutch 2002 → GFS paper 2003 → NDFS 2004 → MapReduce paper 2004 → Nutch+MR 2005 → Hadoop Feb 2006. Named after a toy elephant (not an acronym). Yahoo!: 24,000 nodes by 2009; Apache top-level Jan 2008; 1 TB sort in 209 s on 900 nodes.
- **Why Hadoop:** ~90% of market, scalable, fault tolerant, flexible.
- **NCDC data:** semi-structured, record-oriented, fixed-width ASCII; temp = °C × 10 (-0128 = -12.8 °C); 9999 = missing; preprocess many small station files → one file per year.
- **MapReduce:** 2 phases (map, reduce), key–value in & out; programmer writes map & reduce functions. Map = extract (year, temp) · Shuffle = group by key · Reduce = max. Output: (1949, 111), (1950, 22).
- **Java:** Mapper<LongWritable, Text, Text, IntWritable>; Reducer<Text, IntWritable, Text, IntWritable>; Driver sets paths, classes, types, waitForCompletion. Output file part-r-00000.
- **Scaling out:** job = input + program + config; tasks = map + reduce; scheduled by YARN; input splits (one map per split); data-local → rack-local → off-rack; split size = block size (128 MB).
- **Data flow:** map output → local disk; reducers copy & merge; reduce output → HDFS. Multiple reducers → partition (hash partitioner) → "the shuffle".

CHAPTER 2 · HDFS

- **DFS** = file system managing storage across a network of machines; main challenge: tolerate node failure without data loss.
- **HDFS design:** very large files, streaming access (write once, read many), commodity hardware. **Not for:** low latency (use HBase), lots of small files (NameNode memory), multiple writers/arbitrary modifications.
- **Blocks:** 128 MB default (disk block 512 B; FS block few KB). A 1 MB file uses 1 MB. Benefits: file > any disk; simpler storage; fits replication (3 copies).
- **NameNode** (master): namespace, file tree, metadata; persisted as **namespace image + edit log**. **DataNodes** (workers): store/retrieve blocks, send block reports.
- **Resilience:** write metadata to multiple FS (local + NFS, synchronous & atomic); **Secondary NameNode** merges image + edit log (not a backup).
- **Federation** (2.x): many NameNodes, each manages part of namespace (/user, /share); namespace volume = metadata + block pool; independent.
- **HA:** NameNode = SPOF; cold restart = load image + replay edit log + block reports → 30+ min. Fix: active–standby pair; shared edit log; block reports to both; client failover; standby does checkpoints.

- **File read (6):** open() on DistributedFileSystem → RPC NameNode for block locations (sorted by proximity) → read() on FSDataInputStream/DFSInputStream → stream from closest DataNode → next block → close(). Failures: next closest DN; checksum → other replica + report.
- **File write (7):** create() → RPC NameNode create (checks exists/permissions, else IOException) → write: packets → data queue → DataStreamer → pipeline of 3 DNs → ack queue → close() flushes → complete. Failure: close pipeline, re-queue acks, new block ID, rebuild pipeline.

CHAPTER 3 · Developing MapReduce apps

- **Workflows:** more jobs, not more complex jobs; or use Pig, Hive, Cascading, Crunch, Spark. Mean-max = Job 1 (station, date → max) + Job 2 (station, day-month → mean). Linear chain via JobClient.runJob(); DAG via **JobControl**.
- **MRUnit** + JUnit: MapDriver (1950, −11), ReduceDriver ([10,5] → 10), local job runner for whole job.
- **Job run:** submit() / waitForCompletion(); mapreduce.framework.name = local / classic / yarn.
- **MR1:** Client, JobTracker, TaskTrackers, DFS. Submission (1–4: job ID, check output, compute splits, copy JAR high replication, submitJob) · Initialisation (5–6: queue, one map/split, mapred.reduce.tasks) · Assignment (7: heartbeat) · Execution (8–10: localise JAR, TaskRunner launches new JVM).
- **YARN** (Yet Another Resource Negotiator, 2010, Yahoo!; MR1 bottleneck ~4,000 nodes): JobTracker split into ResourceManager + ApplicationMaster. Entities: Client, RM, NMs, MRAppMaster, DFS. Steps 1–4 submit · 5a/5b launch AM · 6 init · 7 splits · 8 request containers (maps first; reduces after 5%) · 9a/9b start container · 10 YarnChild localises · 11 run.
- **Failures MR1:** task (exception/JVM exit), TaskTracker (no heartbeat; blacklist >4 failed tasks), JobTracker (SPOF → job fails). **YARN:** task (4 attempts), AM (RM restarts it), NM (10 min timeout; AM blacklists >3 failures), RM (checkpoint recovery).
- **Scheduling:** FIFO (default MR1) · Fair (pools per user, fair share over time, preemption) · Capacity (hierarchical queues with capacity, FIFO inside). Preemption = forcefully reassigning resources to higher-priority work.
- **Shuffle & sort:** reducer input sorted by key. Map side: 100 MB circular buffer → spill at 80% → partition → in-memory sort → combiner → merge spills → compress → HTTP. Reduce side: copy (5 threads) → merge ("sort") in rounds → reduce → HDFS.

🧠 All memory tricks in one place

To remember…	Trick	Expands to
Hadoop core components	"HMY body"	HDFS (storage), MapReduce (processing), YARN (manager)
MapReduce phases	"E G S — Every Group Summarised"	Map = Extract, Shuffle = Group, Reduce = Summarise
Data locality	"Home → Neighbour → Other town"	data-local → rack-local → off-rack
HDFS strengths	"L S C"	Large files, Streaming access, Cheap/commodity hardware
HDFS weaknesses	"L S M"	Low latency, Small files, Multiple writers
Library analogy	Catalogue vs shelves	NameNode = catalogue (metadata), DataNodes = shelves (blocks)
File read — 6 steps	"Only Good Readers Really Read Carefully"	Open, Get locations, Read, Read (stream), Read next block, Close
File write — 7 steps	"Cute Cats Write Poems And Close Carefully"	Create, Create (NN), Write, Packet, Ack, Close, Complete
Testing	MapDriver / ReduceDriver / local runner	test mapper → test reducer → test whole job

To remember...	Trick	Expands to
MR1 entities	"Client Just Talks to DFS"	Client, JobTracker, TaskTrackers, DFS
YARN entities	"Can Rahul Not Attend Daily?"	Client, ResourceManager, NodeManagers, ApplicationMaster, DFS
Job run phases (MR1 & YARN)	"Students In An Exam"	Submission, Initialisation, Assignment, Execution
MR1 failures	"Two Tired Jugglers"	Task, TaskTracker, JobTracker
YARN failures	"The Ants Never Rest"	Task, Application master, Node manager, Resource manager
Schedulers	"Friends Play fair" / "Companies Queue up"	Fair → Pools; Capacity → Queues
Shuffle — map side	"Big Spills Please Sweep, Clean, Mop"	Buffer, Spill (80%), Partition, Sort, Combine, Merge
Shuffle — reduce side	"Collect, Mix, Result"	Copy, Merge ("sort"), Reduce

🎯 Important exam questions

Short answers (2–3 marks)

1. What is Hadoop? Who created it? §1.3
2. List the core components of Hadoop. §1.3
3. Why did bigger disks not solve the Big Data problem? §1.2
4. What is data locality optimisation? §1.7
5. Why is the optimal split size equal to the block size? §1.7
6. Define input split, job and task. §1.7
7. What is the default HDFS block size? Why so large? §2.3
8. Where does HDFS not work well? §2.2
9. What is the role of the Secondary NameNode? §2.4
10. What is a namespace volume / block pool? §2.5
11. What is MRUnit? §3.2
12. What does mapreduce.framework.name control? §3.3
13. Expand YARN. Why was it introduced? §3.5
14. What is preemption? §3.7
15. What is a heartbeat? What is blacklisting? §3.4, §3.6

Long answers (5–10 marks)

1. Explain how MapReduce finds the maximum temperature per year, with a data-flow diagram. §1.5
2. Write and explain the Mapper, Reducer and Driver for the max-temperature program. §1.6
3. Explain scaling out: splits, data locality, single and multiple reducer data flow. §1.7
4. Explain the design of HDFS. §2.2
5. Explain HDFS blocks, NameNode and DataNodes with a neat diagram. §2.3–2.4
6. Explain HDFS Federation and High Availability. §2.5–2.6
7. Explain the anatomy of a file read in HDFS. §2.7
8. Explain the anatomy of a file write in HDFS. §2.8
9. Explain MapReduce workflows with an example; explain JobControl. §3.1
10. How do you write a unit test using MRUnit? §3.2
11. Explain how a job runs in classic MapReduce (MR1). §3.4
12. Explain how a job runs in YARN; compare with MR1. §3.5
13. Explain failures in classic MapReduce and in YARN. §3.6
14. Explain the Fair and Capacity schedulers and compare them. §3.7
15. Explain shuffle and sort in MapReduce. §3.8

🎯 THE 6 DIAGRAMS YOU MUST BE ABLE TO DRAW FROM MEMORY

1. MapReduce logical data flow (Fig 1.8)
2. Data flow with multiple reducers (Fig 1.11)
3. HDFS architecture — NameNode, DataNodes, racks (Fig 2.3)
4. Anatomy of a file read & file write (Fig 2.7, 2.8)
5. MR1 and YARN job run (Fig 3.2, 3.4)
6. Shuffle and sort (Fig 3.7)

💡 HOW TO WRITE A 10-MARK ANSWER

1. **Definition** — use the exact textbook words (Definition boxes).
2. **Diagram** — neat, labelled, with **numbered arrows** for step-based topics.
3. **Numbered steps / points** — one per arrow, 1–2 lines each; mention class names.
4. **Failure handling / special cases** — examiners love this extra paragraph.
5. **Example** — the weather dataset fits almost every question.

All the best for CIE-1! 🐘 — Made from your Unit 2 slides (Lectures 1–3) and Tom White, *Hadoop: The Definitive Guide*.